

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN AN TOÀN THÔNG TIN



BÀI TẬP LỚN MÔN: AN TOÀN BẢO MẬT THÔNG TIN

Ngành: Khoa học Máy tính

**ỨNG DỤNG HỌC MÁY TRONG PHÁT HIỆN
TẤN CÔNG BRUTE FORCE TỪ DỮ LIỆU LƯU LƯỢNG
MẠNG**

SỬ DỤNG BỘ DỮ LIỆU CICIDS2017

Giảng viên hướng dẫn: Nguyễn Việt Nhật

NHÓM SINH VIÊN THỰC HIỆN:

STT	Họ và tên sinh viên	Mã số sinh viên	Lớp
1	Nguyễn Việt Tuấn Anh	0204068	67PM2
2	Trần Phùng Đức Anh	0204168	67PM2
3	Nguyễn Vũ Dũng	0205968	67PM2
4	Nguyễn Thu Hương	0209168	67PM2
5	Võ Nguyễn Anh Kiệt	0209968	67PM2

HÀ NỘI, 2026

LỜI CAM ĐOAN

Chúng tôi xin cam đoan rằng bài tập lớn với đề tài: **“ỨNG DỤNG HỌC MÁY TRONG PHÁT HIỆN TẤN CÔNG BRUTE FORCE TỪ DỮ LIỆU LƯU LƯỢNG MẠNG SỬ DỤNG CICIDS2017”** là công trình nghiên cứu và thực nghiệm được nhóm chúng tôi thực hiện một cách nghiêm túc, độc lập dưới sự hướng dẫn của thầy **Nguyễn Việt Nhật**.

Mọi số liệu thực nghiệm, kết quả đo lường và các phát hiện được trình bày trong báo cáo này đều có nguồn gốc minh bạch, được truy xuất trực tiếp từ các artifact đã được kiểm chứng và khóa mã nguồn trong repository công khai của đề tài. Các kết quả dù đạt hiệu năng cao hay suy giảm nghiêm trọng trong các kịch bản thực tế đều được ghi nhận trung thực, không qua chỉnh sửa hay ngụy tạo nhằm mục đích làm đẹp báo cáo.

Các nguồn tài liệu tham khảo, các nghiên cứu đi trước và các công cụ mã nguồn mở được sử dụng đều đã được trích dẫn đầy đủ và rõ ràng theo chuẩn mực học thuật quốc tế. Nhóm chúng tôi hoàn toàn chịu trách nhiệm trước Bộ môn và Nhà trường về tính trung thực của lời cam đoan này.

HÀ NỘI, Tháng 09 năm 2026

Nhóm sinh viên thực hiện

(Ký và ghi rõ họ tên)

Nguyễn Việt Tuấn Anh

Trần Phùng Đức Anh

Nguyễn Vũ Dũng

Nguyễn Thu Hương

Võ Nguyễn Anh Kiệt

LỜI CẢM ƠN

Để hoàn thành bài tập lớn môn học này, chúng em xin bày tỏ lòng biết ơn sâu sắc nhất tới các thầy cô giáo tại TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI, đặc biệt là các thầy cô thuộc KHOA CÔNG NGHỆ THÔNG TIN và BỘ MÔN AN TOÀN THÔNG TIN, những người đã tận tâm truyền đạt những kiến thức nền tảng quý báu về khoa học máy tính, an toàn bảo mật thông tin và mạng máy tính trong suốt quá trình học tập.

Đặc biệt, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới thầy **Nguyễn Việt Nhật**, người đã trực tiếp giảng dạy, định hướng khoa học, tận tình hướng dẫn phương pháp nghiên cứu thực nghiệm và đóng góp những nhận xét chuyên môn quý giá giúp chúng em hoàn thiện bài tập lớn này một cách chuẩn mực và khách quan nhất.

Cuối cùng, chúng em xin gửi lời cảm ơn tới toàn thể các bạn sinh viên trong lớp đã luôn trao đổi, hỗ trợ và chia sẻ tài liệu trong suốt thời gian thực hiện đề tài.

HÀ NỘI, Tháng 09 năm 2026

Nhóm sinh viên thực hiện

*Nguyễn Việt Tuấn Anh, Trần Phùng Đức Anh, Nguyễn Vũ Dũng,
Nguyễn Thu Hương, Võ Nguyễn Anh Kiệt*

TÓM TẮT BÁO CÁO

Trong lĩnh vực an toàn thông tin, ứng dụng học máy trong các hệ thống phát hiện xâm nhập mạng (NIDS) thường đối mặt với hiện tượng đánh giá quá lạc quan khi phân tách dữ liệu ngẫu nhiên. Báo cáo bài tập lớn này thực hiện đánh giá thực nghiệm toàn diện bài toán phát hiện tấn công dò quét mật khẩu (Brute Force gồm `FTP-Patator` và `SSH-Patator`) trên tập dữ liệu CICIDS2017 theo trình tự thời gian, áp dụng kỹ thuật làm sạch ranh giới Purge (60 giây) và Embargo (120 giây) nhằm ngăn ngừa rò rỉ dữ liệu.

Nghiên cứu tập trung đánh giá khả năng tổng quát hóa sang biến thể chưa xuất hiện trong dữ liệu huấn luyện (kịch bản FTP $\rightarrow$ SSH): mô hình chỉ được fit trên tấn công `FTP-Patator` buổi sáng trong tập Train và kiểm thử trên `SSH-Patator` buổi chiều trong tập Test. Đáng lưu ý, tập Validation chứa 1,886 mẫu SSH tham gia vào quá trình chọn cấu hình tối ưu chứ không tham gia fit trọng số mô hình trong benchmark chuẩn; do đó thiết lập này thể hiện khả năng tổng quát hóa từ dữ liệu huấn luyện Train-only chứ không cấu thành quy trình zero-shot nghiêm ngặt hoàn toàn khép kín. Kết quả thực nghiệm trên ba thuật toán dạng cây (Cây quyết định, Rừng ngẫu nhiên, XGBoost) trong kịch bản giữ nguyên cổng dịch vụ cho thấy sự suy giảm hiệu năng nghiêm trọng: Rừng ngẫu nhiên chỉ đạt $F1\text{-Score} = 0.0037$ với tỷ lệ phát hiện SSH đạt 0.19% (7/3,732 luồng), trong khi Cây quyết định và XGBoost hoàn toàn không phát hiện được tấn công SSH ($F1\text{-Score} = 0.0000$). Phân tích khả năng giải thích bằng giá trị SHAP chỉ ra mô hình phụ thuộc áp đảo vào cổng dịch vụ đích (giá trị SHAP trung bình gấp 3.6 lần đặc trưng kế tiếp). Khi bóc tách đặc trưng cổng, tỷ lệ phát hiện SSH tăng nhẹ lên mức 0.19%–0.38% (F1 cao nhất đạt 0.0074 ở Rừng ngẫu nhiên), khẳng định cổng dịch vụ là lối tắt phân loại quan trọng nhưng sự khác biệt hình thái gói tin giữa hai giao thức cũng là yếu tố then chốt tạo nên sự dịch chuyển phân phối dữ liệu.

Trái lại, kịch bản phân tách ngẫu nhiên cho kết quả F1 rất cao (0.9823–0.9997) do tương quan mạnh giữa các luồng trong cùng chiến dịch tấn công. Nghiên cứu cũng phân tích trường hợp điển hình về thiết kế pipeline: việc gộp tập kiểm định vào huấn luyện trước khi tìm kiếm siêu tham số đã vô tình đưa mẫu SSH vào mô hình, làm thay đổi bản chất bài toán và đẩy F1 lên 0.9912. Từ đó, báo cáo đúc kết các bài học thực tiễn về thiết kế quy trình đánh giá khách quan và kiểm soát rò rỉ dữ liệu cho hệ thống phát hiện xâm nhập mạng.

Từ khóa: Hệ thống phát hiện xâm nhập (NIDS), CICIDS2017, Brute Force, Phân tách theo thời gian, Rò rỉ dữ liệu, Tổng quát hóa biến thể, SHAP.

ABSTRACT

In information security, applying machine learning (ML) to Network Intrusion Detection Systems (NIDS) has garnered significant attention. However, empirical studies employing random data partitioning frequently report near-perfect F1-scores ($> 99\%$), introducing substantial optimistic evaluation bias and concealing risks when models encounter evolving traffic patterns in operational environments.

This course project report presents an empirical evaluation of brute force attack detection (FTP-Patator and SSH-Patator) on the standard CICIDS2017 Tuesday dataset. By implementing a time-based split combined with conservative boundary purging ($q = 60\text{s}$) and embargo ($g = 120\text{s}$) techniques, the evaluation investigates **FTP-trained Cross-Subtype Generalization**: models are fitted exclusively on morning FTP-Patator attacks and subsequently evaluated on afternoon SSH-Patator attacks. Importantly, the validation split contains 1,886 SSH samples and participates in model and hyperparameter selection; hence, this setup assesses unseen-subtype generalization from training data rather than a strict zero-shot pipeline.

Empirical results across three tree-based algorithms (Decision Tree, Random Forest, and XGBoost) demonstrate severe performance degradation on the unobserved attack subtype: in the primary benchmark (time/with_port), the representative model (Random Forest) achieves an F1-score of 0.0037 with an SSH detection rate of 0.19% (capturing only 7 out of 3,732 SSH flows), while Decision Tree and XGBoost yield an F1-score of 0.0000. Model explainability via SHAP and decision tree inspection shows strong model reliance on service ports (Destination Port exhibits the highest mean absolute SHAP value, 3.6 times greater than the second feature). When service ports are removed via port ablation, the SSH detection rate increases slightly to 0.19%–0.38% (with the highest time-based F1-score reaching 0.0074 for Random Forest without_port), indicating that while the destination port serves as a primary shortcut, packet-level statistical differences between FTP and SSH also contribute to the observed distribution shift.

Conversely, under stratified random splitting controls, models attain high F1-scores of 0.9823–0.9997, attributable to strong autocorrelation among flows belonging to the same attack campaign (within-campaign similarity). Furthermore, we analyze a practical MLOps case study wherein a pipeline design flaw—concatenating Train and Validation splits prior to hyperparameter search combined with default scikit-learn `refit=True` behavior—re-trains

the estimator on the combined data containing 1,886 validation SSH flows, shifting the evaluation regime to supervised learning and causing the test F1-score to rise from 0.0000 to 0.9912. This work synthesizes methodological lessons regarding temporal boundary control and realistic empirical evaluation for ML-based NIDS.

Keywords: Network Intrusion Detection Systems (NIDS), CICIDS2017, Brute Force Attacks, Temporal Split, Optimistic Evaluation Bias, Cross-Subtype Generalization, Model Explainability (SHAP).

MỤC LỤC

Lời Cam Đoan	i
Lời Cảm Ơn	ii
Tóm Tắt Báo Cáo	iii
Abstract	iv
Danh Mục Ký Hiệu và Chữ Viết Tắt	x
1 Giới Thiệu và Đặt Vấn Đề	1
1.1 Bối cảnh an ninh mạng và nguy cơ tấn công brute force	1
1.2 Học máy trong phát hiện xâm nhập dựa trên luồng mạng	1
1.3 Vấn đề đánh giá: Độ lệch đánh giá và yếu tố thời gian	2
1.4 Mục tiêu và câu hỏi nghiên cứu	2
1.5 Đóng góp thực nghiệm của bài tập lớn	3
1.6 Cấu trúc của báo cáo	4
2 Cơ Sở Lý Thuyết	5
2.1 Bản chất lưu lượng mạng và cơ chế phân tích luồng	5
2.1.1 Khái niệm luồng mạng (IP Network Flow)	5
2.1.2 Cơ chế sinh đặc trưng của CICFlowMeter	5
2.1.3 Phân loại Flow đã hoàn tất và NIDS thời gian thực	6
2.2 Cơ chế tấn công Brute Force trên giao thức FTP và SSH	6
2.2.1 Tấn công Brute Force đối với FTP (Cổng 21)	6
2.2.2 Tấn công Brute Force đối với SSH (Cổng 22)	6
2.3 Các thuật toán học máy dạng cây	7

2.3.1	Cây quyết định (Decision Tree)	7
2.3.2	Rừng ngẫu nhiên (Random Forest)	7
2.3.3	XGBoost	7
2.3.4	Khả năng giải thích mô hình với SHAP	8
2.4	Đánh giá mô hình trên dữ liệu mất cân bằng	8
2.4.1	Hệ thống chỉ số từ ma trận nhầm lẫn	8
2.4.2	Nghịch lý độ chính xác trong NIDS	9
2.4.3	Đường cong Precision-Recall (PR) và Average Precision (AP)	9
3	Tập Dữ Liệu và Định Nghĩa Bài Toán	10
3.1	Tổng Quan Tập Dữ Liệu CICIDS2017 Tuesday	10
3.2	Nguồn Gốc Dữ Liệu và Xác Minh Tính Toàn Vẹn SHA-256	10
3.3	Phục Hồi Dòng Thời Gian 24 Giờ (Timestamp Reconstruction)	11
3.4	Định Nghĩa Bài Toán và Không Gian Nhấn	11
3.5	Quy Trình Làm Sạch Dữ Liệu Thô	12
3.6	Dòng Thời Gian Tấn Công Thực Tế và Phân Bỏ Mẫu	12
4	Phương Pháp và Kiến Trúc Hệ Thống	14
4.1	Kiến trúc tổng thể của hệ thống thực nghiệm	14
4.2	Chính sách quản lý đặc trưng	14
4.2.1	Thực nghiệm bóc tách cổng dịch vụ	15
4.3	Thiết kế chiến lược phân tách dữ liệu	15
4.3.1	Cơ chế Purge và Embargo chống rò rỉ ranh giới thời gian	15
4.3.2	Chiến lược đối chứng: Phân tách ngẫu nhiên	16
4.4	Quy trình tiền xử lý độc lập trên tập train	16
4.5	Nguyên tắc đóng băng mô hình	17
5	Thiết Kế Thực Nghiệm	18
5.1	Môi Trường Thực Thi Phần Cứng và Phần Mềm	18
5.2	Kiểm Soát Tính Ngẫu Nhiên	18

5.3	Xử Lý Mất Cân Bằng Lớp Trên Tập Huấn Luyện	18
5.4	Thiết Lập Không Gian Tìm Kiếm Siêu Tham Số	19
5.5	Tiêu Chí Lựa Chọn Mô Hình trên Tập Validation	20
5.6	Phân định rõ hai chế độ thực nghiệm	20
5.6.1	Chế độ 1: Đánh giá khả năng tổng quát hóa sang biến thể mới (kích bản FTP → SSH)	20
5.6.2	Chế độ 2: Huấn luyện lại trên tập gộp (Nghiên cứu điển hình về thiết kế pipeline)	21
5.7	Đảm bảo tính toàn vẹn và khả năng tái lập	21
6	Kết Quả Thực Nghiệm	22
6.1	Tổng hợp thống kê dữ liệu sau tiền xử lý	22
6.2	Kết quả đánh giá và lựa chọn mô hình trên tập kiểm định	22
6.3	Kết quả đánh giá tập kiểm thử theo thời gian	24
6.4	Kết quả thực nghiệm đối chứng trên phân tách ngẫu nhiên	25
6.4.1	Đo lường định lượng độ lệch đánh giá	26
6.5	Kết quả thực nghiệm bóc tách công mạng	27
6.6	Phân tích khả năng giải thích	28
6.6.1	Phân tích tầm quan trọng đặc trưng bằng SHAP	28
6.6.2	Trích xuất cấu trúc cây quyết định	29
6.7	Nghiên cứu điển hình: Tác động của thiết kế pipeline và cờ refit trên XGBoost	31
6.7.1	Truy vết nguyên nhân kỹ thuật trong mã nguồn	31
6.7.2	Hậu quả đối với bản chất bài toán đánh giá	32
7	Kết Luận và Hướng Phát Triển	34
7.1	Tổng kết kết quả và trả lời các câu hỏi nghiên cứu	34
7.2	Đánh giá tính hợp lệ và các giới hạn của đề tài	35
7.3	Hướng phát triển và mở rộng	36
7.4	Bảng phân công nhiệm vụ và đóng góp của thành viên	37

Tài Liệu Tham Khảo	39
A Hướng Dẫn Cài Đặt và Tái Lập Thực Nghiệm	41
A.1 Yêu cầu môi trường và cài đặt phụ thuộc	41
A.2 Chuẩn bị dữ liệu và phân tách	41
A.3 Chạy thực nghiệm và kiểm thử tự động	42
B Không Gian Siêu Tham Số và Cấu Hình Tối Ưu	43
C Danh Mục Đầy Đủ Các Đặc Trưng Mạng	44
D Quy Tắc Rẽ Nhánh Của Cây Quyết Định	48

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Quy ước biểu diễn số liệu:

Trong toàn bộ báo cáo này, các giá trị định lượng được biểu diễn thống nhất theo quy chuẩn khoa học quốc tế: sử dụng dấu chấm (.) làm dấu phân cách phần thập phân (ví dụ: 0.7181, 99.97%) và dấu phẩy (,) làm dấu phân cách các lớp hàng nghìn (ví dụ: 140,425 luồng, 445,888 bản ghi).

Ký hiệu / Viết tắt	Ý nghĩa giải thích đầy đủ
AP	Average Precision (Độ chính xác trung bình có trọng số trên dải ngưỡng phân loại)
CIC	Canadian Institute for Cybersecurity (Viện An ninh mạng Canada)
CSV	Comma-Separated Values (Định dạng tệp dữ liệu phân tách bằng dấu phẩy)
DPI	Deep Packet Inspection (Kỹ thuật phân tích gói tin sâu)
DT	Decision Tree (Cây quyết định)
EDA	Exploratory Data Analysis (Khám phá và phân tích dữ liệu ban đầu)
FN	False Negative (Số lượng mẫu tấn công bị phân loại nhầm là bình thường)
FP	False Positive (Số lượng mẫu bình thường bị cảnh báo nhầm là tấn công)
FPR	False Positive Rate (Tỷ lệ báo động giả trên tổng số lưu lượng bình thường)
FTP	File Transfer Protocol (Giao thức truyền tệp tin, Cổng mặc định 21)
IAT	Inter-Arrival Time (Khoảng thời gian giữa các gói tin liên tiếp trong một flow)
i.i.d.	Independent and Identically Distributed (Độc lập và cùng phân phối xác suất)
IDS	Intrusion Detection System (Hệ thống phát hiện xâm nhập)
IP	Internet Protocol (Giao thức mạng Internet)
IPS	Intrusion Prevention System (Hệ thống ngăn ngừa xâm nhập mạng)
ML	Machine Learning (Học máy)
MLOps	Machine Learning Operations (Quy trình vận hành và triển khai học máy)
NIDS	Network Intrusion Detection System (Hệ thống phát hiện xâm nhập mạng)
PR	Precision-Recall (Đường cong tương quan giữa Độ chính xác và Độ thu hồi)
RF	Random Forest (Rừng ngẫu nhiên)

Ký hiệu / Viết tắt	Ý nghĩa giải thích đầy đủ
ROC	Receiver Operating Characteristic (Đường cong đặc trưng hoạt động của bộ thu)
RQ	Research Question (Câu hỏi nghiên cứu)
SHA-256	Secure Hash Algorithm 256-bit (Thuật toán mã hóa băm an toàn 256-bit)
SHAP	SHapley Additive exPlanations (Phương pháp giải thích dự đoán dựa trên lý thuyết trò chơi)
SMOTE	Synthetic Minority Over-sampling Technique (Kỹ thuật tổng hợp mẫu thiểu số nhân tạo)
SSH	Secure Shell (Giao thức truyền thông an toàn và điều khiển từ xa, Cổng mặc định 22)
TCP	Transmission Control Protocol (Giao thức điều khiển truyền vận)
TN	True Negative (Số lượng mẫu bình thường được phân loại đúng)
TP	True Positive (Số lượng mẫu tấn công được phát hiện chính xác)
XGB	eXtreme Gradient Boosting (Thuật toán tăng cường độ dốc cực đại)

DANH SÁCH BẢNG

3.1	Quy tắc ánh xạ nhãn bài toán phân loại nhị phân phát hiện tấn công dò quét mật khẩu.	12
3.2	Thống kê phân bố dữ liệu các tập Train, Validation và Test theo chiến lược Temporal Split v1.	13
4.1	Chính sách kiểm soát không gian đặc trưng và danh mục đặc trưng hằng số bị loại bỏ.	15
6.1	Kết quả đánh giá và lựa chọn cấu hình siêu tham số tối ưu trên tập Validation (Mốc 10:02 – 14:28).	23
6.2	Hiệu năng tổng thể trên tập Test đối chuẩn giữa 12 tổ hợp thực nghiệm (Đánh giá sau khi khóa cấu hình).	24
6.3	Độ lệch hiệu năng ($\Delta F1$) giữa Phân tách Ngẫu nhiên và Phân tách Thời gian trên tập Validation.	26
6.4	Thực nghiệm Bóc tách Cổng mạng (Port Ablation) trên tập Validation phân tách theo thời gian.	27
6.5	Top 10 đặc trưng mạng có đóng góp lớn nhất vào quyết định phân loại theo giá trị SHAP trung bình.	29
6.6	Đối chiếu hiệu năng của XGBoost giữa chế độ đánh giá tổng quát hóa chuẩn mực và chế độ tiếp xúc validation do thiết kế pipeline (Refit).	33
7.1	Tổng hợp các mối đe dọa đến tính hợp lệ của nghiên cứu và giải pháp kiểm soát.	36
7.2	Bảng phân công nhiệm vụ và tỷ lệ đóng góp của các thành viên trong nhóm.	38
B.1	Chi tiết tham số tối ưu của 12 mô hình được lựa chọn trên tập Validation.	43

DANH SÁCH HÌNH VẼ

6.1	Ma trận nhầm lẫn chi tiết của các mô hình trên tập kiểm thử phân tách theo thời gian, thể hiện trực quan tỷ lệ True Positive rất thấp đối với biến thể tấn công SSH.	25
6.2	Đối chiếu hiệu năng giữa phân tách ngẫu nhiên và phân tách theo thời gian, trực quan hóa mức độ chênh lệch hiệu năng do tương quan nội bộ chiến dịch.	26
6.3	Đường cong ROC và Precision-Recall đối chiếu giữa các mô hình và kịch bản thực nghiệm, phản ánh sự biến thiên diện tích dưới đường cong khi có và không có đặc trưng cổng mạng.	28
6.4	Biểu đồ phân bố giá trị SHAP (SHAP Summary Plot) của Top các đặc trưng quan trọng trên 500 mẫu kiểm định.	30
6.5	Cấu trúc 3 tầng đầu tiên của Cây Quyết Định minh họa ngưỡng rẽ nhánh ưu tiên tại <code>Destination Port ≤ 21.5</code>	30

Chương 1

GIỚI THIỆU VÀ ĐẶT VẤN ĐỀ

1.1 Bối cảnh an ninh mạng và nguy cơ tấn công brute force

Trong hạ tầng công nghệ thông tin hiện đại, an toàn an ninh thông tin là yêu cầu thiết yếu đối với các tổ chức. Trong số các hình thức tấn công mạng phổ biến, tấn công dò quét mật khẩu (Brute Force) nhắm vào các giao thức quản trị và truyền tệp từ xa như SSH (Cổng 22) và FTP (Cổng 21) vẫn là mối đe dọa thường trực. Bằng cách tự động hóa các lượt thử thông tin xác thực, kẻ tấn công có thể chiếm đoạt quyền điều khiển máy chủ nội bộ và tạo bàn đạp mở rộng tấn công trong mạng.

Các hệ thống phát hiện xâm nhập mạng (NIDS) truyền thống dựa trên luật/chữ ký như Snort [1] hay Suricata [2] hoạt động dựa trên đối khớp chuỗi nhị phân (signatures) hoặc các quy tắc ngưỡng tĩnh. Mặc dù có ưu điểm về tốc độ đối với các mẫu đã biết, cách tiếp cận này gặp khó khăn trước các biến thể tấn công mới, chi phí duy trì luật thủ công lớn và đặc biệt bị hạn chế trước các giao thức mã hóa tầng ứng dụng như SSH, nơi nội dung tải trọng (payload) được bảo vệ bằng mật mã.

1.2 Học máy trong phát hiện xâm nhập dựa trên luồng mạng

Để giải quyết bài toán trên, hướng tiếp cận dựa trên học máy kết hợp phân tích luồng mạng được nghiên cứu rộng rãi. Thay vì phân tích gói tin chuyên sâu (DPI) tốn kém tài nguyên và bị cản trở bởi mã hóa, phương pháp luồng mạng chỉ trích xuất các đặc trưng thống kê tổng hợp của cả phiên truyền thông như: tổng thời lượng phiên (Flow Duration), số lượng gói tin hai chiều, thống kê kích thước gói tin, thời gian trễ giữa các gói tin liên tiếp (IAT) và các cờ điều khiển TCP.

Các thuật toán học máy dạng cây (Decision Tree, Random Forest, XGBoost) thường được lựa chọn trong bài toán này nhờ tốc độ huấn luyện/suy luận nhanh, khả năng xử lý tốt dữ liệu dạng bảng và khả năng trích xuất các quy tắc phân loại rõ ràng.

1.3 Vấn đề đánh giá: Độ lệch đánh giá và yếu tố thời gian

Mặc dù nhiều công bố học thuật báo cáo điểm số F1 rất cao ($> 99\%$) cho các mô hình NIDS, việc áp dụng thực tế vẫn gặp nhiều thách thức. Sommer và Paxson [3] đã nhấn mạnh sự khác biệt căn bản của an ninh mạng: môi trường mạng liên tục biến động và kẻ tấn công chủ động thay đổi hành vi.

Gần đây, Arp và cộng sự [4] đã chỉ ra nhiều cạm bẫy thực nghiệm phổ biến trong học máy cho an ninh mạng, nổi bật là **độ lệch đánh giá quá lạc quan** (*optimistic evaluation bias*) khi áp dụng chiến lược **phân chia dữ liệu ngẫu nhiên**.

Khi áp dụng phân chia ngẫu nhiên trên tập dữ liệu mạng như CICIDS2017 [5]:

- Các flow thuộc cùng một chiến dịch tấn công (phát sinh từ cùng một công cụ, cùng địa chỉ IP nguồn, cùng cấu hình thử nghiệm) có thể bị phân bổ đồng thời vào cả tập Huấn luyện (Train), Kiểm định (Validation) và Kiểm thử (Test).
- Sự tương quan mạnh nội tại giữa các mẫu trong cùng chiến dịch (*within-campaign similarity*) có thể khiến giả thiết các quan sát độc lập và phân phối đồng nhất (*i.i.d.*) không còn được đảm bảo chặt chẽ.
- Do đó, mô hình có thể đạt điểm số cao nhờ việc nhận dạng lại dấu hiệu thống kê của chiến dịch đã thấy (mang tính chất nội suy) thay vì năng lực tổng quát hóa sang chiến dịch mới.

1.4 Mục tiêu và câu hỏi nghiên cứu

Bài tập lớn này tập trung xây dựng một quy trình thực nghiệm khách quan, kiểm soát chặt chẽ rò rỉ thông tin theo thời gian trên tập dữ liệu CICIDS2017 Tuesday, nhằm khảo sát năng lực thực tế của các mô hình học máy dạng cây khi đối mặt với biến thể tấn công chưa xuất hiện trong tập huấn luyện.

Đề tài giải quyết 4 câu hỏi nghiên cứu (RQ) sau:

- RQ1:** (*Khả năng tổng quát hóa sang biến thể chưa học theo thời gian*) Khi mô hình dạng cây được huấn luyện trên dữ liệu Train chỉ chứa FTP-Patator, khả năng phát hiện SSH-Patator xuất hiện ở giai đoạn thời gian sau thay đổi như thế nào?
- RQ2:** (*Mức độ ảnh hưởng của phép chia ngẫu nhiên*) Phép chia ngẫu nhiên (Random Split) tạo ra sự khác biệt hiệu năng (Optimistic Evaluation Bias) ở mức độ định lượng bao nhiêu so với phép chia phân tách theo thứ tự thời gian (Time-based Split)?

- RQ3:** (*Vai trò của cổng dịch vụ và các đặc trưng hành vi*) Cổng dịch vụ đích (`Destination Port`) đóng vai trò như thế nào trong dự đoán của mô hình? Khi loại bỏ cổng mạng (`Port Ablation`), các đặc trưng thống kê hành vi gói tin có đủ để nhận diện lưu lượng tấn công hay không?
- RQ4:** (*Tác động của thiết kế pipeline và giao thức huấn luyện lại*) Lỗi thiết kế pipeline khi chủ động gộp `Train` và `Validation` trước khi truyền vào tìm kiếm siêu tham số kết hợp với cờ `refit=True` làm thay đổi bản chất bài toán đánh giá từ tổng quát hóa sang học có giám sát như thế nào?

1.5 Đóng góp thực nghiệm của bài tập lớn

Các kết quả thực nghiệm chính của bài tập lớn bao gồm:

- Xây dựng quy trình thực nghiệm kiểm soát thời gian:** Thực hiện tiền xử lý và phục hồi thứ tự thời gian 24h; áp dụng kỹ thuật làm sạch ranh giới bảo thủ `Purge` ($q = 60s$) và `Embargo` ($g = 120s$) loại bỏ 7,632 dòng ranh giới; thiết lập bài toán đánh giá khả năng tổng quát hóa sang biến thể mới với mô hình fit trên `Train-only` (kịch bản `FTP` → `SSH`) trên dữ liệu `CICIDS2017 Tuesday`, trong đó tập `Validation` chứa 1,886 mẫu `SSH` chỉ tham gia vào việc chọn cấu hình tối ưu chứ không tham gia fit trọng số mô hình trong benchmark chuẩn.
- Đánh giá thực nghiệm 3 mô hình học máy dạng cây:** Đối chuẩn `Decision Tree`, `Random Forest` và `XGBoost` qua 12 tổ hợp thực nghiệm ($3 \text{ mô hình} \times 2 \text{ chiến lược chia} \times 2 \text{ kịch bản cổng}$). Ghi nhận trung thực sự suy giảm hiệu năng của mô hình trên tập `Test` theo thời gian: trong kịch bản chính có cổng mạng, `Random Forest` đạt $F1\text{-Score} = 0.0037$, nhận diện được 0.19% tấn công `SSH` (7/3,732 flow).
- Phân tích cơ chế suy luận qua SHAP và cấu trúc cây:** Sử dụng `SHAP TreeExplainer` và phân tích nhánh rẽ của cây quyết định để làm rõ việc mô hình phụ thuộc mạnh vào cổng dịch vụ (`Destination Port` có giá trị `SHAP` trung bình lớn nhất 0.1237, gấp 3.6 lần đặc trưng thứ hai).
- Phân tích case study MLOps thực tế về cờ refit:** Làm rõ trường hợp pipeline chủ động gộp `Train` và `Validation` trước khi truyền vào `RandomizedSearchCV.fit()` kết hợp với cờ `refit=True` mặc định của `scikit-learn`, khiến mô hình được huấn luyện lại trên toàn bộ tập gộp (chứa 1,886 mẫu `SSH` từ `Validation`), dẫn đến việc điểm `Test F1` tăng từ 0.0000 lên 0.9912.
- Tổ chức mã nguồn và số liệu có tính tái lập cao:** Toàn bộ số liệu trong báo cáo được truy xuất tự động từ các artifact đã đóng băng, đi kèm mã băm `SHA-256` xác minh tệp

nguồn, với khả năng tái lập được kiểm soát thông qua cấu hình môi trường đã ghi nhận và kiểm chứng (xem Phụ lục [A](#)).

1.6 Cấu trúc của báo cáo

Báo cáo được tổ chức thành 7 chương:

- **Chương 1:** Giới thiệu bối cảnh an ninh mạng, vấn đề khoa học, mục tiêu và 4 câu hỏi nghiên cứu.
- **Chương 2:** Cơ sở lý thuyết về luồng mạng, cơ chế tấn công Brute Force, các thuật toán học máy dạng cây và hệ thống chỉ số đánh giá mất cân bằng lớp.
- **Chương 3:** Chi tiết tập dữ liệu CICIDS2017 Tuesday, mã băm SHA-256, quy tắc phục hồi thời gian và quy trình làm sạch dữ liệu.
- **Chương 4:** Phương pháp đề xuất và kiến trúc hệ thống, chính sách đặc trưng, kỹ thuật Purge/Embargo và nguyên tắc đồng băng cấu hình.
- **Chương 5:** Thiết kế thực nghiệm, không gian siêu tham số và tiêu chí lựa chọn cấu hình mô hình trên tập kiểm định.
- **Chương 6:** Kết quả và phân tích thực nghiệm qua 12 tổ hợp, giải thích SHAP, cấu trúc rẽ nhánh cây và nghiên cứu điển hình về cờ refit.
- **Chương 7:** Kết luận, trả lời 4 câu hỏi nghiên cứu, đánh giá tính hợp lệ, hướng phát triển tương lai và bảng phân công nhiệm vụ thành viên.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Bản chất lưu lượng mạng và cơ chế phân tích luồng

2.1.1 Khái niệm luồng mạng (IP Network Flow)

Theo chuẩn RFC 7011 (IPFIX) [6], một luồng mạng (Flow) được định nghĩa là một tập hợp các gói tin (packets) hoặc khung truyền (frames) đi qua một điểm quan sát (Observation Point) trong một khoảng thời gian nhất định và chia sẻ chung một bộ thuộc tính nhận dạng đặc trưng (Flow Keys). Trong thực tế và trong bối cảnh bài tập lớn này, bộ 5 thuộc tính truyền thống (Traditional 5-Tuple) là cách nhận dạng flow phổ biến được sử dụng:

$$\text{Flow 5-Tuple} = \langle \text{Source IP, Destination IP, Source Port, Destination Port, Protocol} \rangle \quad (2.1)$$

2.1.2 Cơ chế sinh đặc trưng của CICFlowMeter

Công cụ CICFlowMeter [5] thực hiện tổng hợp các gói tin thuộc cùng một 5-Tuple theo cả hai hướng truyền thông (Forward - từ client đến server, và Backward - từ server về client) để tạo thành một luồng hai chiều (Bi-directional Flow). Một flow được coi là kết thúc khi xảy ra một trong các điều kiện sau:

1. Xuất hiện cờ điều khiển đóng kết nối TCP (FIN hoặc RST).
2. Khoảng thời gian không có gói tin mới vượt quá ngưỡng chờ không hoạt động (Inactivity Timeout, mặc định 15 đến 60 giây).
3. Tổng thời lượng của flow vượt quá thời gian tối đa cho phép (Flow Timeout).

Sau khi flow đóng, CICFlowMeter tính toán các chỉ số thống kê tổng hợp mô tả toàn bộ vòng đời của flow, bao gồm thời lượng phiên truyền, thống kê kích thước gói tin (min, max, mean, std), thời gian trễ giữa các gói tin liên tiếp (Flow IAT, Fwd IAT, Bwd IAT), các cờ điều khiển TCP (SYN, ACK, PSH, URG, FIN) và kích thước cửa sổ nhận ban đầu (Init_Win_bytes).

2.1.3 Phân loại Flow đã hoàn tất và NIDS thời gian thực

Một điểm cốt lõi cần làm rõ về mặt lý thuyết: vì toàn bộ các đặc trưng thống kê nói trên chỉ được tính toán đầy đủ **sau khi flow đã kết thúc hoàn toàn**, các mô hình học máy được xây dựng trên dữ liệu này hoạt động ở cơ chế **Phân loại Flow đã hoàn tất (Completed-Flow Classification)**.

Hệ thống này **không phải** là hệ thống phát hiện xâm nhập thời gian thực ở mức gói tin. Mô hình không can thiệp ngăn chặn tấn công ngay ở những gói tin bắt tay đầu tiên và không đo lường độ trễ phát hiện theo mili-giây.

2.2 Cơ chế tấn công Brute Force trên giao thức FTP và SSH

2.2.1 Tấn công Brute Force đối với FTP (Cổng 21)

FTP là giao thức truyền tệp hoạt động trên kênh văn bản rõ (plain-text) ở tầng ứng dụng:

- Quá trình xác thực diễn ra thông qua các cặp lệnh USER và PASS.
- Máy chủ phản hồi bằng mã trạng thái 230 User logged in (thành công) hoặc 530 Login incorrect (thất bại).
- Khi công cụ Patator (ftp_patator) thực hiện tấn công dò quét mật khẩu, nó thiết lập liên tục các kết nối TCP mới, gửi lệnh thử mật khẩu và đóng kết nối ngay khi nhận mã 530.
- **Đặc trưng thống kê:** Vì là văn bản rõ, các gói tin yêu cầu và phản hồi có kích thước ngắn và tương đối cố định. Lưu lượng hướng đi (Fwd) thường có độ lệch chuẩn kích thước gói tin nhỏ.

2.2.2 Tấn công Brute Force đối với SSH (Cổng 22)

SSH là giao thức truyền thông an toàn cung cấp kênh xác thực và điều khiển được bảo vệ bằng mật mã:

- Trước khi quá trình gửi thông tin xác thực diễn ra, client và server trải qua pha bắt tay mật mã: trao đổi phiên bản giao thức, thương lượng thuật toán mã hóa đối xứng, trao đổi khóa và xác thực khóa công khai của host.
- Quá trình xác thực mật khẩu diễn ra hoàn toàn bên trong kênh đã được bảo vệ.
- **Đặc trưng thống kê:** Pha trao đổi khóa mật mã làm phát sinh các gói tin có kích thước lớn hơn đáng kể so với FTP. Cấu trúc bắt tay khiến các chỉ số về kích thước gói tin

trung bình (Packet Length Mean) và thời gian trễ giữa các gói tin (Flow IAT) của SSH có phân phối thống kê khác biệt so với FTP.

2.3 Các thuật toán học máy dạng cây

2.3.1 Cây quyết định (Decision Tree)

Cây quyết định phân loại (Decision Tree [7]) phân chia không gian đặc trưng thành các vùng siêu hình hộp rời rạc thông qua một tập hợp các quy tắc rẽ nhánh dạng bậc thang ($x_j \leq \theta$).

Tại mỗi nút phân chia, thuật toán tìm kiếm đặc trưng j và ngưỡng θ tối ưu nhằm tối đa hóa mức giảm độ vẩn đục (Impurity Reduction). Chỉ số vẩn đục phổ biến là **Gini Impurity**:

$$I_G(p) = 1 - \sum_{k=0}^1 p_k^2 \quad (2.2)$$

trong đó p_k là tỷ lệ mẫu thuộc lớp k tại nút đang xét. Ưu điểm nổi bật của Decision Tree là tính minh bạch và khả năng trực quan hóa cấu trúc rẽ nhánh, giúp giải thích logic phân loại. Tuy nhiên, Decision Tree đơn lẻ có phương sai cao và dễ bị quá khớp (overfitting).

2.3.2 Rừng ngẫu nhiên (Random Forest)

Random Forest [8] là phương pháp học kết hợp dựa trên kỹ thuật đóng bao (bagging). Mô hình xây dựng một tập hợp gồm B cây quyết định độc lập:

$$\hat{y} = \text{mode} \{T_1(\mathbf{x}), T_2(\mathbf{x}), \dots, T_B(\mathbf{x})\} \quad (2.3)$$

Hai cơ chế ngẫu nhiên hóa cốt lõi của Random Forest giúp giảm phương sai mà không làm tăng độ chệch:

1. **Bootstrap Sampling**: Mỗi cây được huấn luyện trên một tập dữ liệu con được rút mẫu có hoàn lại từ tập huấn luyện gốc.
2. **Random Feature Subspace**: Tại mỗi nút rẽ nhánh, thuật toán chỉ chọn ngẫu nhiên một tập con gồm $m \approx \sqrt{D}$ đặc trưng để chọn ra điểm phân chia tốt nhất, giúp giải tương quan giữa các cây.

2.3.3 XGBoost

XGBoost [9] là thuật toán nâng cao thuộc họ Gradient Boosting. Boosting xây dựng các cây quyết định tuần tự, trong đó mỗi cây mới $f_t(\mathbf{x})$ được tối ưu để bù đắp sai số của tổ hợp các

cây trước đó:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i) \quad (2.4)$$

Hàm mục tiêu được tối thiểu hóa tại bước t bao gồm hàm mất mát khả vi và số hạng chính quy hóa (regularization term) để kiểm soát độ phức tạp của cây:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l\left(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\right) + \Omega(f_t) \quad (2.5)$$

với số hạng phạt:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (2.6)$$

trong đó T là số lượng lá, w_j là trọng số tại lá j , γ là ngưỡng phạt tối thiểu để tạo lá mới và λ là hệ số chính quy hóa L2.

2.3.4 Khả năng giải thích mô hình với SHAP

Để giải thích đóng góp của từng đặc trưng vào dự đoán của các mô hình dạng cây, phương pháp SHAP [10] dựa trên lý thuyết trò chơi hợp tác Shapley được sử dụng. Giá trị SHAP gán cho mỗi đặc trưng đại diện cho mức độ thay đổi kỳ vọng dự đoán khi đặc trưng đó xuất hiện so với khi vắng mặt:

$$\phi_i(x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f_x(S \cup \{i\}) - f_x(S)] \quad (2.7)$$

Phương pháp TreeExplainer cho phép tính toán chính xác giá trị SHAP trên các mô hình dạng cây với độ phức tạp đa thức, giúp xác định các đặc trưng có ảnh hưởng lớn nhất đến quyết định phân loại.

2.4 Đánh giá mô hình trên dữ liệu mất cân bằng

2.4.1 Hệ thống chỉ số từ ma trận nhầm lẫn

Trong bài toán phát hiện xâm nhập mạng, lớp tấn công (Attack, nhãn 1) thường chiếm tỷ lệ nhỏ so với lưu lượng bình thường (Normal, nhãn 0). Ma trận nhầm lẫn gồm 4 đại lượng: True Positive (TP), False Positive (FP), False Negative (FN) và True Negative (TN).

Các chỉ số phái sinh chuẩn:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (2.8)$$

$$\text{False Positive Rate (FPR)} = \frac{FP}{FP + TN} \quad (2.9)$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \quad (2.10)$$

2.4.2 Nghịch lý độ chính xác trong NIDS

Chỉ số Accuracy tổng thể:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.11)$$

có thể gây hiểu nhầm nghiêm trọng nếu được sử dụng độc lập trên dữ liệu mất cân bằng. Giả sử tập kiểm thử có 97.4% mẫu bình thường và 2.6% mẫu tấn công (như tập Test CICIDS2017 Tuesday). Một bộ phân loại tầm thường luôn dự đoán mọi flow là bình thường ($TP = 0, FP = 0$) sẽ đạt ngay **Accuracy = 97.4%** mặc dù không phát hiện được bất kỳ cuộc tấn công nào (**Recall = 0.0%**). Do đó, F1-Score, đường cong Precision-Recall và Average Precision là các chỉ số đánh giá cần thiết.

2.4.3 Đường cong Precision-Recall (PR) và Average Precision (AP)

Như Davis và Goadrich [11] đã chỉ ra, trên các tập dữ liệu mất cân bằng lớp, đường cong ROC có thể tạo ra cảm nhận quá lạc quan do đại lượng TN trong mẫu số của FPR rất lớn. Đường cong Precision-Recall biểu diễn sự đánh đổi giữa Precision và Recall khi thay đổi ngưỡng phân loại $\tau \in [0, 1]$.

Theo đặc tả của scikit-learn [12], chỉ số **Average Precision (AP)** được tính bằng giá trị trung bình có trọng số của Precision theo mức tăng Recall giữa các ngưỡng:

$$\text{AP} = \sum_n (R_n - R_{n-1}) P_n \quad (2.12)$$

trong đó P_n và R_n là Precision và Recall tại ngưỡng thứ n . AP đánh giá chất lượng xếp hạng (ranking) các mẫu dương trên dải ngưỡng phân loại; AP không đồng nhất hoàn toàn với diện tích tích phân hình thang thô của PR-AUC và không đo lường độ chuẩn xác xác suất (probability calibration).

Chương 3

TẬP DỮ LIỆU VÀ ĐỊNH NGHĨA BÀI TOÁN

3.1 Tổng Quan Tập Dữ Liệu CICIDS2017 Tuesday

Tập dữ liệu CICIDS2017 [5], [13] được xây dựng bởi Viện An ninh mạng Canada (CIC), Đại học New Brunswick (UNB). Đây là một trong những bộ dữ liệu chuẩn mực và được sử dụng rộng rãi để nghiên cứu và đánh giá các hệ thống phát hiện xâm nhập mạng.

Tập dữ liệu ghi lại lưu lượng mạng trong 5 ngày làm việc liên tiếp (từ thứ Hai 03/07/2017 đến thứ Sáu 07/07/2017) trên hạ tầng mạng mô phỏng gồm mạng nạn nhân và mạng tấn công bên ngoài. Trong đó, ngày thứ Ba (Tuesday, 04/07/2017) được dành riêng để thực hiện các kịch bản tấn công dò quét mật khẩu (Brute Force):

- **Tấn công buổi sáng:** `FTP-Patator` thực hiện dò quét mật khẩu dịch vụ FTP (Cổng 21) từ 09:20 đến 10:20.
- **Tấn công buổi chiều:** `SSH-Patator` thực hiện dò quét mật khẩu dịch vụ SSH (Cổng 22) từ 14:00 đến 15:00.

3.2 Nguồn Gốc Dữ Liệu và Xác Minh Tính Toàn Vẹn SHA-256

Dữ liệu thô được sử dụng trong bài tập lớn là tệp CSV trích xuất chính thức từ gói `GeneratedLabelledFlows.zip` của UNB:

- **Đường dẫn:** `data/raw/Tuesday-WorkingHours.pcap_ISCX.csv`
- **Mã băm SHA-256 đã xác minh:**

`ae9c88e10c41a8eb1ff454ae98bc513454925097d0b0b57180f94e79de445815`

- **Tổng số dòng thô ban đầu:** 445,909 bản ghi flow (tương ứng 445,910 dòng trên đĩa tính cả 1 dòng tiêu đề).

Việc lưu trữ và kiểm tra mã băm SHA-256 cho phép xác minh tệp đầu vào có cùng nội dung byte với tệp đã ghi nhận mã băm, hỗ trợ tính tái lập thực nghiệm trên cùng tệp dữ liệu nguồn.

3.3 Phục Hồi Dòng Thời Gian 24 Giờ (Timestamp Reconstruction)

Trong quá trình khám phá dữ liệu (EDA), chúng tôi nhận thấy trong tệp Tuesday được sử dụng, các timestamp quan sát được không đủ thông tin để phân biệt một số giờ buổi sáng/buổi chiều theo dòng thời gian kỳ vọng:

- Định dạng chuỗi thời gian thô được lưu dưới dạng `dd/MM/yyyy` kết hợp với `h:mm` hoặc `hh:mm`.
- Các giá trị giờ thô quan sát được chỉ gồm hai nhóm:

$$\mathcal{H}_{\text{raw}} = \{1, 2, 3, 4, 5, 8, 9, 10, 11, 12\} \quad (3.1)$$

mà không có hậu tố chỉ định AM/PM.

- Các bản ghi diễn ra vào buổi chiều (từ 13:00 đến 17:00) xuất hiện dưới dạng 1:00 đến 5:00.

Để phục hồi đúng thứ tự thời gian phục vụ phép phân chia theo thời gian (Time-based Split), dự án áp dụng quy tắc tái dựng thời gian `reconstruct_timestamp()` tại mã nguồn `scripts/prepare_splits.py`:

$$H_{24h} = \begin{cases} H_{\text{raw}} + 12 & \text{nếu } H_{\text{raw}} \in \{1, 2, 3, 4, 5\} \\ H_{\text{raw}} & \text{nếu } H_{\text{raw}} \in \{8, 9, 10, 11, 12\} \end{cases} \quad (3.2)$$

Nguyên tắc kiểm soát rò rỉ dữ liệu (No-Leakage Policy): Đây là một quy tắc tái dựng của pipeline dự án, được thiết lập hoàn toàn độc lập với nhãn tấn công (`Label`) và cổng đích (`Destination Port`), chỉ dựa trên phân tích dòng thời gian của toàn bộ traffic trong ngày Tuesday (thời gian làm việc từ 8:30 sáng đến 17:00 chiều).

3.4 Định Nghĩa Bài Toán và Không Gian Nhãn

Bài tập lớn định nghĩa bài toán phân loại là **phân loại nhị phân** nhằm nhận diện lưu lượng bất thường:

$$y = \begin{cases} 0 & \text{nếu flow là lưu lượng bình thường (BENIGN)} \\ 1 & \text{nếu flow là tấn công Brute Force (FTP-Patator hoặc SSH-Patator)} \end{cases} \quad (3.3)$$

Chi tiết ánh xạ nhãn được trình bày trong Bảng 3.1.

Bảng 3.1: Quy tắc ánh xạ nhãn bài toán phân loại nhị phân phát hiện tấn công dò quét mật khẩu.

Nhãn gốc trong dữ liệu	Nhãn y	Xử lý bài toán	Mô tả bản chất an ninh mạng
BENIGN	0	Lớp thường (Normal)	Lưu lượng mạng của các tác vụ và giao dịch hợp lệ của người dùng nội bộ.
FTP-Patator	1	Lớp tấn công (Attack)	Hành vi dò quét thử mật khẩu liên tục đối với dịch vụ FTP (Cổng đích 21).
SSH-Patator	1	Lớp tấn công (Attack)	Hành vi dò quét thử mật khẩu tự động đối với dịch vụ SSH (Cổng đích 22).

Mặc dù bài toán phân loại chính là nhị phân, thông tin chi tiết về dạng tấn công (Subtype) vẫn được bảo toàn trong một cột siêu dữ liệu riêng biệt để theo dõi tỷ lệ phát hiện chi tiết từng dạng tấn công (Subtype Recall).

3.5 Quy Trình Làm Sạch Dữ Liệu Thô

Tập dữ liệu thô trải qua các bước làm sạch kỹ thuật:

- Loại bỏ flow có thời lượng không hợp lệ:** Loại bỏ 17 bản ghi có Flow Duration ≤ 0 hoặc bị thiếu (NaN).
- Loại bỏ bản ghi trùng lặp:** Loại bỏ 4 dòng bản ghi trùng lặp hoàn toàn trên tất cả các cột dữ liệu.
- Loại bỏ cột đặc trưng trùng lặp:** Cột Fwd Header Length.1 có giá trị trùng lặp 100% với cột Fwd Header Length được loại bỏ khỏi không gian đặc trưng.
- Xử lý giá trị vô cùng:** Có 327 giá trị vô cùng ($\pm\infty$, chủ yếu phát sinh do phép chia cho thời lượng bằng 0 trong các cột Flow Bytes/s và Flow Packets/s) được chuyển đổi thành giá trị NaN để xử lý qua bộ điền giá trị thiếu (Imputer).

Sau khi làm sạch kỹ thuật, tổng số bản ghi hợp lệ đưa vào quy trình phân tách là 445,888 dòng.

3.6 Dòng Thời Gian Tấn Công Thực Tế và Phân Bỏ Mẫu

Bảng 3.2 tổng hợp chi tiết số lượng mẫu và phân bỏ nhãn qua các tập sau khi áp dụng chiến lược phân tách theo thời gian (được trình bày chi tiết ở Chương 4).

Phân tích dòng thời gian cho thấy đặc điểm phân bố tự nhiên của dữ liệu CICIDS2017 Tuesday:

Bảng 3.2: Thống kê phân bố dữ liệu các tập Train, Validation và Test theo chiến lược Temporal Split v1.

Tập dữ liệu	Tổng số flow	BENIGN	Tấn công	FTP-Patator	SSH-Patator	Tỷ lệ Attack (%)
Train	90,273	85,139	5,134	5,134	0	5.69%
Validation	203,826	199,522	4,304	2,418	1,886	2.11%
Test	144,157	140,425	3,732	0	3,732	2.59%
Purge/Embargo	7,632	6,967	665	386	279	8.71%
Tổng số hợp lệ	445,888	432,053	13,835	7,938	5,897	3.10%

- **Tập Train** ($e < 10 : 00$): Chứa 85,139 mẫu bình thường và 5,134 mẫu FTP-Patator. **Không có mẫu SSH-Patator nào (0 mẫu).**
- **Tập Validation** ($10 : 02 \leq s$ và $e < 14 : 30$): Chứa đoạn kết của chiến dịch FTP (2,418 mẫu) và giai đoạn đầu của chiến dịch SSH (1,886 mẫu).
- **Tập Test** ($s \geq 14 : 32$): Chứa 140,425 mẫu bình thường và 3,732 mẫu SSH-Patator. **Không có mẫu FTP-Patator nào (0 mẫu).**

Đặc tính phân bố này tạo ra bài toán đánh giá khả năng **tổng quát hóa sang biến thể mới (kịch bản FTP $\rightarrow$ SSH)**. Trong thiết lập này, mô hình hoàn toàn không được huấn luyện trên mẫu SSH-Patator nào trong tập Train; tập Validation chứa 1,886 mẫu SSH tham gia vào quá trình lựa chọn siêu tham số, trước khi mô hình được đánh giá độc lập trên tập Test chỉ chứa biến thể SSH.

Chương 4

PHƯƠNG PHÁP VÀ KIẾN TRÚC HỆ THỐNG

4.1 Kiến trúc tổng thể của hệ thống thực nghiệm

Nhằm đảm bảo tính khách quan và kiểm soát nguy cơ rò rỉ dữ liệu, hệ thống thực nghiệm được thiết kế theo kiến trúc đường ống gồm 5 giai đoạn độc lập:

1. **Giai đoạn 1: Chuẩn bị dữ liệu và phân tách:** Áp dụng phức hồi dòng thời gian và thuật toán Purge/Embargo để chia tách dữ liệu thành Train, Validation và Test độc lập.
2. **Giai đoạn 2: Tiền xử lý chỉ học từ tập Train:** Loại bỏ các đặc trưng hằng số và khớp (fit) bộ điền giá trị thiếu (median) duy nhất trên $\mathcal{D}_{\text{train}}$.
3. **Giai đoạn 3: Tinh chỉnh siêu tham số:** Tìm kiếm cấu hình tối ưu của 3 mô hình cây trên không gian $\mathcal{D}_{\text{train}} \times \mathcal{D}_{\text{val}}$.
4. **Giai đoạn 4: Đóng băng cấu hình:** Khóa tham số và cấu hình tốt nhất vào tệp siêu dữ liệu `frozen.json`.
5. **Giai đoạn 5: Mở kiểm thử một lần duy nhất:** Đánh giá hiệu năng tổng quát hóa trên $\mathcal{D}_{\text{test}}$ mà không tinh chỉnh lại bất kỳ tham số nào.

4.2 Chính sách quản lý đặc trưng

Để ngăn chặn hiện tượng học lỗi tắt [14], các trường thông tin định danh và siêu dữ liệu không phản ánh hành vi mạng thuần túy đều bị loại bỏ khỏi không gian đặc trưng:

- **Các trường cấm sử dụng:** Flow ID, Source IP, Destination IP, Source Port, Timestamp, Timestamp_fixed, Protocol, TimeBin10, _row_id, index. Việc giữ lại địa chỉ IP hay nhãn thời gian sẽ khiến mô hình ghi nhớ máy chủ tấn công thay vì học mẫu lưu lượng.

Chi tiết chính sách quản lý đặc trưng được trình bày trong Bảng 4.1.

Bảng 4.1: Chính sách kiểm soát không gian đặc trưng và danh mục đặc trưng hằng số bị loại bỏ.

Nhóm đặc trưng	Chi tiết các trường thông tin và quy tắc xử lý
Trường cấm sử dụng	Flow ID, Source IP, Destination IP, Source Port, Timestamp, Timestamp_fixed, Protocol, TimeBin10, _row_id, index.
Lý do cấm	Ngăn chặn mô hình học thuộc lòng địa chỉ máy chủ tấn công hoặc ghi nhớ thời điểm diễn ra chiến dịch.
10 Đặc trưng hằng số	Bwd PSH Flags, Fwd URG Flags, Bwd URG Flags, CWE Flag Count, Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, Bwd Avg Bulk Rate.
Cơ chế phát hiện	Toàn bộ giá trị có phương sai bằng 0 trên tập Train ($\mathcal{D}_{\text{train}}$) $\rightarrow$ Loại bỏ đồng bộ trên Val và Test.
Không gian cuối cùng	67 đặc trưng (kịch bản with_port) và 66 đặc trưng (kịch bản without_port).

4.2.1 Thực nghiệm bóc tách cổng dịch vụ

Cổng dịch vụ đích (Destination Port) có giá trị bằng 21 cho giao thức FTP và 22 cho giao thức SSH. Để trả lời câu hỏi nghiên cứu RQ3 xem mô hình có bị phụ thuộc thái quá vào số cổng hay không, nhóm thiết kế hai nhánh không gian đặc trưng song song:

1. **Nhánh có cổng** (with_port, 67 đặc trưng): Bao gồm trường Destination Port kết hợp với toàn bộ các đặc trưng thống kê hành vi luồng mạng.
2. **Nhánh không có cổng** (without_port, 66 đặc trưng): Loại bỏ hoàn toàn trường Destination Port, buộc mô hình chỉ dựa vào các đặc trưng thống kê gói tin như kích thước, khoảng thời gian IAT, cờ TCP và độ dài tiêu đề gói tin.

Danh mục định nghĩa chi tiết của toàn bộ 67 đặc trưng mạng được cung cấp tại Phụ lục C.

4.3 Thiết kế chiến lược phân tách dữ liệu

4.3.1 Cơ chế Purge và Embargo chống rò rỉ ranh giới thời gian

Trong phân tích chuỗi thời gian mạng, một flow có thời điểm bắt đầu s và thời lượng d (tính bằng micro-giây) sẽ kéo dài qua một khoảng thời gian. Để tính toán thời điểm hoàn tất một

cách bảo thủ, nhóm lựa chọn bổ sung thêm sai số làm tròn $q = 60s$:

$$e = s + d + q \quad (4.1)$$

Tại mỗi mốc thời gian cắt T_{cutoff} , nguy cơ rò rỉ thông tin xảy ra khi:

1. **Flow vượt ngang ranh giới (Boundary Crossing):** Flow bắt đầu trước T_{cutoff} nhưng kết thúc sau T_{cutoff} ($s < T_{\text{cutoff}}$ và $e \geq T_{\text{cutoff}}$). Nếu giữ lại, thông tin của tương lai đã rò rỉ vào quá khứ. Giải pháp là **Purge** (loại bỏ hoàn toàn các flow này).
2. **Flow diễn ra ngay sau ranh giới:** Để tránh các ảnh hưởng trễ của các kết nối TCP vừa đóng, một khoảng đệm an toàn (Embargo buffer) $g = 120s$ được áp dụng ngay sau mốc cắt. Mọi flow bắt đầu trong khoảng $[T_{\text{cutoff}}, T_{\text{cutoff}} + g)$ đều bị loại bỏ.

Việc lựa chọn các tham số $q = 60s$ và $g = 120s$ là quyết định thiết kế bảo thủ nhằm hạn chế tối đa rủi ro flow kéo dài hoặc lưu lượng phản hồi trễ vi phạm ranh giới phân tách.

Với hai mốc cắt thời gian được xác định từ trước:

- **Mốc A:** 2017-07-04 10:00:00 (Phân cách Train và Validation)
- **Mốc B:** 2017-07-04 14:30:00 (Phân cách Validation và Test)

Thuật toán Purge và Embargo đã loại bỏ tổng cộng **7,632 dòng** ranh giới không an toàn (xem Bảng 3.2), giúp phân tách ranh giới thời gian rõ ràng giữa các tập dữ liệu.

4.3.2 Chiến lược đối chứng: Phân tách ngẫu nhiên

Để làm cơ sở đối chứng đo lường hiện tượng độ lệch đánh giá quá lạc quan (*optimistic evaluation bias*), một phân vùng đối chứng được phân chia ngẫu nhiên theo phương pháp phân tầng (Stratified Random Split) với tỷ lệ số lượng mẫu tương ứng: Train (20.6%), Validation (46.5%) và Test (32.9%). Phép chia này phân bổ ngẫu nhiên các flow mà không xét đến thứ tự thời gian. Trong bối cảnh dữ liệu mạng, cách tiếp cận này có thể tạo ra độ lệch đánh giá quá lạc quan do các flow có tương quan mạnh từ cùng một chiến dịch tấn công (tương quan nội bộ chiến dịch – *within-campaign similarity*) được phân bổ đồng thời vào cả ba tập, dẫn đến một đánh giá mang tính chất nội suy (*interpolation-like evaluation*) thay vì phản ánh năng lực khái quát hóa thực tế.

4.4 Quy trình tiền xử lý độc lập trên tập train

Một nguyên tắc quan trọng trong thiết kế đường ống MLOps là **bộ tiền xử lý không được học tham số từ dữ liệu tương lai**. Toàn bộ các bước học tham số tiền xử lý được thực hiện

trên tập $\mathcal{D}_{\text{train}}$ theo quy trình chuẩn hóa:

1. **Loại bỏ đặc trưng hằng số:** Thuật toán kiểm tra phương sai của từng đặc trưng trên $\mathcal{D}_{\text{train}}$. Có 10 đặc trưng có giá trị không đổi (phương sai bằng 0) trên $\mathcal{D}_{\text{train}}$ (bao gồm các cờ hiểm như Bwd PSH Flags, Fwd URG Flags, CWE Flag Count và các trường bulk). Danh sách 10 cột này được lưu lại và áp dụng loại bỏ đồng bộ trên $\mathcal{D}_{\text{val}}$ và $\mathcal{D}_{\text{test}}$.
2. **Điền giá trị thiếu (Missing Imputation):** Khởi tạo `SimpleImputer` với chiến lược trung vị (`strategy='median'`). Giá trị trung vị của từng đặc trưng được tính toán duy nhất trên $\mathcal{D}_{\text{train}}$, sau đó được dùng để điền các giá trị NaN trên cả ba tập $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{val}}$ và $\mathcal{D}_{\text{test}}$.

4.5 Nguyên tắc đóng băng mô hình

Để hạn chế nguy cơ tinh chỉnh siêu tham số lặp đi lặp lại trên tập Test (rò rỉ thông tin tập kiểm thử qua quá trình thử nghiệm), nhóm áp dụng quy tắc mở kiểm thử một lần duy nhất (*Single-Open Test Protocol*):

- Toàn bộ quá trình thử nghiệm, tìm kiếm siêu tham số và lựa chọn mô hình đại diện được thực hiện hoàn toàn trên tập $\mathcal{D}_{\text{train}}$ và $\mathcal{D}_{\text{val}}$.
- Cấu hình tốt nhất sau đó được xuất ra tệp cấu hình `frozen.json` và tính mã băm SHA-256 xác thực.
- Quy trình đánh giá trên tập kiểm thử kiểm tra cờ xác thực `TEST_OPENED.json`. Tập kiểm thử chỉ được mở đúng một lần duy nhất để ghi nhận chỉ số cuối cùng mà không tinh chỉnh lại mô hình hay tham số.

Chương 5

THIẾT KẾ THỰC NGHIỆM

5.1 Môi Trường Thực Thi Phần Cứng và Phần Mềm

Toàn bộ các thực nghiệm trong bài tập lớn được triển khai trên môi trường máy trạm độc lập với các thông số kỹ thuật được kiểm soát và ghi nhận:

- **Phần cứng máy trạm:** Vi xử lý đa nhân kiến trúc x86_64, bộ nhớ RAM 16 GB, hỗ trợ tập lệnh SIMD AVX2.
- **Hệ điều hành:** Microsoft Windows 11 (64-bit).
- **Môi trường thực thi thực tế:** Python 3.12 cùng hệ thống thư viện khoa học dữ liệu đã kiểm chứng gồm `numpy==1.26.4`, `pandas==2.2.2`, `scikit-learn==1.6.1`, `xgboost==3.0.5`, `shap==0.48.0` và `matplotlib==3.9.2`.
- **Tương thích dự án:** Mã nguồn khai báo dải phụ thuộc `requires-python = ">=3.11,<3.13"` trong `pyproject.toml`, trong đó gói giải thích mô hình SHAP được quản lý dưới dạng thành phần tùy chọn (`pip install -e ".[xai]"`).

5.2 Kiểm Soát Tính Ngẫu Nhiên

Để đảm bảo khả năng tái lập có kiểm soát với cấu hình thực nghiệm đã ghi nhận và đối chiếu, toàn bộ các tiến trình sinh số giả ngẫu nhiên từ quá trình phân chia dữ liệu, lấy mẫu bootstrap của Random Forest, khởi tạo trọng số của XGBoost và lấy mẫu nền của SHAP đều được cố định với một hạt giống ngẫu nhiên:

$$\text{random_state} = 42 \quad (5.1)$$

5.3 Xử Lý Mất Cân Bằng Lớp Trên Tập Huấn Luyện

Như đã phân tích tại Bảng 3.2, trong tập $\mathcal{D}_{\text{train}}$ theo thời gian, số lượng mẫu bình thường là 85,139 mẫu trong khi số lượng mẫu tấn công chỉ có 5,134 mẫu (tỷ lệ tấn công là 5.69%). Tỷ lệ mất cân bằng lớp xấp xỉ 1 : 16.58.

Để khắc phục hiện tượng mô hình bị thiên lệch về lớp đa số mà không làm biến dạng phân phối không gian mẫu (như các phương pháp tổng hợp dữ liệu SMOTE thường gây nhiễu ranh giới mạng), dự án áp dụng kỹ thuật gán trọng số lớp tính toán duy nhất từ $\mathcal{D}_{\text{train}}$:

- **Decision Tree và Random Forest:**

Sử dụng tham số trọng số `class_weight='balanced'`:

$$w_c = \frac{N_{\text{train}}}{2 \cdot N_{\text{train},c}} \implies w_0 = \frac{90,273}{2 \times 85,139} \approx 0.53, \quad w_1 = \frac{90,273}{2 \times 5,134} \approx 8.79 \quad (5.2)$$

- **Đối với XGBoost:** Sử dụng tham số `scale_pos_weight`:

$$\text{scale_pos_weight} = \frac{N_{\text{train, normal}}}{N_{\text{train, attack}}} = \frac{85,139}{5,134} \approx 16.5834 \quad (5.3)$$

5.4 Thiết Lập Không Gian Tìm Kiếm Siêu Tham Số

Để đảm bảo tính khách quan và khả năng đối chiếu, không gian siêu tham số được chuẩn hóa thành lưới tìm kiếm chính thức và được khóa cấu hình trong tệp lưu vết `artifacts/week3_week4/frozen.json`. Các tham số quan trọng nhất của từng thuật toán được xác định như sau:

1. Cây Quyết Định (Decision Tree):

- Tiêu chuẩn phân tách (`criterion`): Mặc định `'gini'` (đã khảo sát bổ sung `'entropy'` trong các thử nghiệm thăm dò).
- Độ sâu tối đa (`max_depth`): `[6, 8, 12, None]` (với `None` là cây phát triển hoàn toàn đến khi lá thuần nhất).
- Số mẫu tối thiểu tại nút lá (`min_samples_leaf`): `[5, 10]`.

2. Rừng Ngẫu Nhiên (Random Forest):

- Số lượng cây (`n_estimators`): `[100, 150, 300]`.
- Độ sâu tối đa (`max_depth`): `[10, 16, None]`.
- Số mẫu tối thiểu tại nút lá (`min_samples_leaf`): `[5]`.
- Số đặc trưng tại mỗi nút (`max_features`): `'sqrt'`.

3. XGBoost:

- Số lượng cây (`n_estimators`): `[100, 200]`.

- Độ sâu cây (`max_depth`): [3, 4, 6].
- Tốc độ học (`learning_rate`): 0.08.
- Tỷ lệ lấy mẫu đặc trưng (`colsample_bytree`): 0.8.
- Tỷ lệ lấy mẫu dòng (`subsample`): 0.8.

Bên cạnh lưới chính thức trên, các thực nghiệm mở rộng (như khảo sát 50 lượt lặp với công cụ `RandomizedSearchCV` cho Decision Tree hoặc khảo sát tốc độ học [0.05, 0.08, 0.1] cho XGBoost) được ghi nhận độc lập tại các thư mục thực nghiệm riêng lẻ để phục vụ phân tích độ nhạy. Chi tiết cấu hình tối ưu được lựa chọn cho 12 mô hình được tổng hợp tại Phụ lục B.

5.5 Tiêu Chí Lựa Chọn Mô Hình trên Tập Validation

Trên tập kiểm định $\mathcal{D}_{\text{val}}$, các cấu hình được đánh giá theo thứ tự ưu tiên:

1. **Ưu tiên 1:** Điểm **F1-Score** trên lớp tấn công (tại ngưỡng mặc định $\tau = 0.5$).
2. **Ưu tiên 2:** Điểm **Average Precision (AP)** trên đường cong Precision-Recall nếu F1 tương đương.
3. **Ưu tiên 3:** Tỷ lệ báo động giả **FPR** thấp hơn nếu cả F1 và AP đều xấp xỉ nhau.

Mô hình đạt điểm số tối ưu trên $\mathcal{D}_{\text{val}}$ sẽ được lựa chọn làm đại diện của kịch bản đó để chuyển sang bước đánh giá trên tập kiểm thử $\mathcal{D}_{\text{test}}$.

5.6 Phân định rõ hai chế độ thực nghiệm

Nhằm đảm bảo tính minh bạch khoa học và tránh việc diễn giải sai lệch kết quả, bài tập lớn phân định rõ ràng **hai chế độ thực nghiệm** độc lập đã được thực hiện:

5.6.1 Chế độ 1: Đánh giá khả năng tổng quát hóa sang biến thể mới (kịch bản FTP $\rightarrow$ SSH)

Đây là giao thức đánh giá trung tâm của bài tập lớn:

- Mô hình được fit duy nhất trên tập $\mathcal{D}_{\text{train}}$ (chỉ chứa FTP-Patator, không chứa mẫu SSH-Patator nào).
- Tập $\mathcal{D}_{\text{val}}$ chứa 1,886 mẫu SSH-Patator và được sử dụng để đánh giá phân loại, phục vụ lựa chọn cấu hình siêu tham số tốt nhất (mô hình không được huấn luyện trên tập này trong benchmark chuẩn).

- Thiết lập này thể hiện khả năng **tổng quát hóa sang biến thể chưa xuất hiện trong dữ liệu huấn luyện** (*unseen-subtype generalization from Train-only fitting*), kèm lưu ý phương pháp luận rằng sự hiện diện của SSH trong $\mathcal{D}_{\text{val}}$ hỗ trợ việc lựa chọn mô hình nên đây không phải là một quy trình zero-shot nghiêm ngặt hoàn toàn khép kín từ đầu đến cuối.
- Sau khi chọn được tham số tốt nhất, mô hình đại diện giữ nguyên trọng số đã học từ $\mathcal{D}_{\text{train}}$ để đánh giá độc lập trên tập $\mathcal{D}_{\text{test}}$ (chỉ chứa SSH-Patator).

5.6.2 Chế độ 2: Huấn luyện lại trên tập gộp (Nghiên cứu điển hình về thiết kế pipeline)

Đây là chế độ thực nghiệm phát sinh từ thiết kế pipeline trong script tuning mở rộng:

- Thiết kế pipeline: Train và Validation được chủ động gộp thành $\mathbf{X}_{\text{combined}} = \mathbf{X}_{\text{train}} \cup \mathbf{X}_{\text{val}}$ (thông qua lệnh `pd.concat([X_train, X_val])`) trước khi truyền vào hàm tìm kiếm siêu tham số.
- Thư viện scikit-learn [15] hoạt động đúng đặc tả với cờ mặc định `refit=True` (khi không bật cờ `--strict-train-only`): sau khi tìm kiếm tham số tối ưu bằng phân tách tùy biến, mô hình tốt nhất được huấn luyện lại trên toàn bộ dữ liệu đầu vào đã truyền vào (**toàn bộ tập gộp $\mathbf{X}_{\text{combined}}$**).
- Vì $\mathbf{X}_{\text{val}}$ có chứa 1,886 mẫu SSH-Patator, mô hình trong chế độ này được huấn luyện có giám sát trên các mẫu SSH trước khi bước vào tập Test.
- Chế độ này được sử dụng như một nghiên cứu điển hình về MLOps tại Mục 6.7 (Chương 6) để làm rõ ảnh hưởng của thiết kế pipeline và hành vi của thư viện đối với tính hợp lệ của bài toán đánh giá.

5.7 Đảm bảo tính toàn vẹn và khả năng tái lập

Nhằm kiểm tra tính toàn vẹn và ngăn ngừa lỗi hồi quy trong đường ống xử lý dữ liệu, các ràng buộc về mốc thời gian, kỹ thuật Purge/Embargo và tính đóng băng của tập Test được tự động hóa kiểm định thông qua bộ kiểm thử hồi quy `tests/test_protocol.py` (quy trình thực thi chi tiết xem tại Phụ lục A).

Chương 6

KẾT QUẢ THỰC NGHIỆM

Chương này trình bày các kết quả định lượng thu được từ 12 tổ hợp thực nghiệm (gồm 3 mô hình học máy dạng cây $\times$ 2 chiến lược phân tách dữ liệu $\times$ 2 kịch bản không gian đặc trưng), được tổ chức theo các nội dung sau:

6.1 Tổng hợp thống kê dữ liệu sau tiền xử lý

Sau khi hoàn tất quy trình tiền xử lý chỉ học từ tập Train độc lập, không gian đặc trưng cuối cùng được xác định rõ ràng:

- Số lượng đặc trưng ban đầu: 77 đặc trưng mạng (sau khi đã loại bỏ các trường định danh và cột trùng lặp `Fwd Header Length.1`).
- Số lượng đặc trưng hằng số bị loại bỏ: 10 đặc trưng có phương sai bằng 0 trên $\mathcal{D}_{\text{train}}$.
- Kịch bản có cổng (`with_port`): **67 đặc trưng**.
- Kịch bản loại bỏ cổng (`without_port`): **66 đặc trưng**.

Về số lượng mẫu (xem lại Bảng 3.2):

- **Tập Huấn luyện ($\mathcal{D}_{\text{train}}$):** 90,273 dòng, gồm 85,139 mẫu bình thường và 5,134 mẫu FTP-Patator (tỷ lệ tấn công 5.69%). Không có mẫu SSH-Patator nào.
- **Tập Kiểm định ($\mathcal{D}_{\text{val}}$):** 203,826 dòng, gồm 199,522 mẫu bình thường và 4,304 mẫu tấn công (2,418 mẫu FTP-Patator và 1,886 mẫu SSH-Patator, tỷ lệ tấn công 2.11%).
- **Tập Kiểm thử ($\mathcal{D}_{\text{test}}$):** 144,157 dòng, gồm 140,425 mẫu bình thường và 3,732 mẫu SSH-Patator (tỷ lệ tấn công 2.59%). Không có mẫu FTP-Patator nào.

6.2 Kết quả đánh giá và lựa chọn mô hình trên tập kiểm định

Quá trình tìm kiếm siêu tham số trên tập kiểm định $\mathcal{D}_{\text{val}}$ mang lại các kết quả định lượng chi tiết được tổng hợp trong Bảng 6.1. Không gian siêu tham số và bộ tham số tối ưu được khóa

của toàn bộ 12 kịch bản mô hình được thống kê đầy đủ tại Phụ lục B. Trong thiết lập này, tập $\mathcal{D}_{\text{val}}$ chứa 2,418 mẫu FTP-Patator và 1,886 mẫu SSH-Patator, được sử dụng thuần túy để tính toán các độ đo phục vụ việc lựa chọn cấu hình mô hình tối ưu; mô hình hoàn toàn không được cập nhật trọng số trên tập này trong benchmark chuẩn mực.

Bảng 6.1: Kết quả đánh giá và lựa chọn cấu hình siêu tham số tối ưu trên tập Validation (Mốc 10:02 – 14:28).

Mô hình	Phân tích	Cổng	F1-Score	Precision	Recall	AP	ROC-AUC	FPR
Decision Tree	Thời gian	Có cổng	0.7170	0.9971	0.5597	0.5690	0.7798	3.51×10^{-5}
Random Forest	Thời gian	Có cổng	0.7181	1.0000	0.5602	0.7326	0.8892	0.00
XGBoost	Thời gian	Có cổng	0.7181	1.0000	0.5602	0.6045	0.8921	0.00
Decision Tree	Thời gian	Không cổng	0.7169	0.9967	0.5597	0.5676	0.7798	4.01×10^{-5}
Random Forest	Thời gian	Không cổng	0.7134	0.9821	0.5602	0.5694	0.7759	2.21×10^{-4}
XGBoost	Thời gian	Không cổng	0.7136	0.9841	0.5597	0.6874	0.9704	1.95×10^{-4}
Decision Tree	Ngẫu nhiên	Có cổng	0.9939	0.9900	0.9978	0.9970	0.9989	3.23×10^{-4}
Random Forest	Ngẫu nhiên	Có cổng	0.9997	1.0000	0.9994	0.9998	0.9999	0.00
XGBoost	Ngẫu nhiên	Có cổng	0.9995	0.9992	0.9998	0.9999	1.0000	2.49×10^{-5}
Decision Tree	Ngẫu nhiên	Không cổng	0.9799	0.9662	0.9941	0.9895	0.9970	0.0011
Random Forest	Ngẫu nhiên	Không cổng	0.9942	0.9930	0.9953	0.9993	0.9999	2.24×10^{-4}
XGBoost	Ngẫu nhiên	Không cổng	0.9984	0.9984	0.9983	0.9999	1.0000	4.97×10^{-5}

Trong kịch bản đánh giá chính theo thời gian có cổng:

1. **Cây Quyết Định (Decision Tree):** Đạt F1-Score = 0.7170, Precision = 0.9971, Recall = 0.5597, Average Precision = 0.5690 và FPR = 3.51×10^{-5} với cấu hình gồm max_depth=6 và min_samples_leaf=5.
2. **Rừng Ngẫu Nhiên (Random Forest):** Đạt F1-Score = **0.7181**, Precision = **1.0000**, Recall = **0.5602**, Average Precision = **0.7326** và FPR = **0.00** với cấu hình n_estimators=100, max_depth=16, min_samples_leaf=5. Mô hình bắt được 99.71% mẫu FTP-Patator nhưng không phát hiện được mẫu SSH-Patator nào (0.00%).
3. **XGBoost:** Đạt F1-Score = 0.7181, Precision = 1.0000, Recall = 0.5602, Average Precision = 0.6045 và FPR = 0.00 với cấu hình n_estimators=200, max_depth=4, learning_rate=0.08.

Quyết định lựa chọn mô hình đại diện: Mặc dù Random Forest và XGBoost đạt điểm F1 xấp xỉ ngang nhau (0.7181) và cùng có tỷ lệ báo động giả cực thấp ($\text{FPR} \leq 10^{-5}$), Random Forest có chỉ số Average Precision cao hơn (Average Precision = **0.7326** so với 0.6045 của XGBoost). Điều này cho thấy chất lượng xếp hạng (ranking) các mẫu dương trên dải ngưỡng phân loại của Random Forest tốt hơn theo độ đo AP.

Do đó, theo thứ tự ưu tiên lựa chọn đã thiết lập, **Random Forest (time/with_port)** được chọn là mô hình đại diện ưu tú nhất của kịch bản chính để làm trọng tâm phân tích sâu

và giải thích suy luận qua SHAP, bên cạnh việc đối chuẩn đồng thời cả ba thuật toán trên tập kiểm thử $\mathcal{D}_{\text{test}}$.

6.3 Kết quả đánh giá tập kiểm thử theo thời gian

Sau khi đóng băng cấu hình siêu tham số và lưu vết mã băm trong `frozen.json`, quy trình thực nghiệm tiến hành mở tập kiểm thử $\mathcal{D}_{\text{test}}$ đúng một lần duy nhất. Kết quả đánh giá của 3 mô hình trên tập kiểm thử theo thời gian (kịch bản chính theo thời gian có công) được thể hiện tại Phần A của Bảng 6.2.

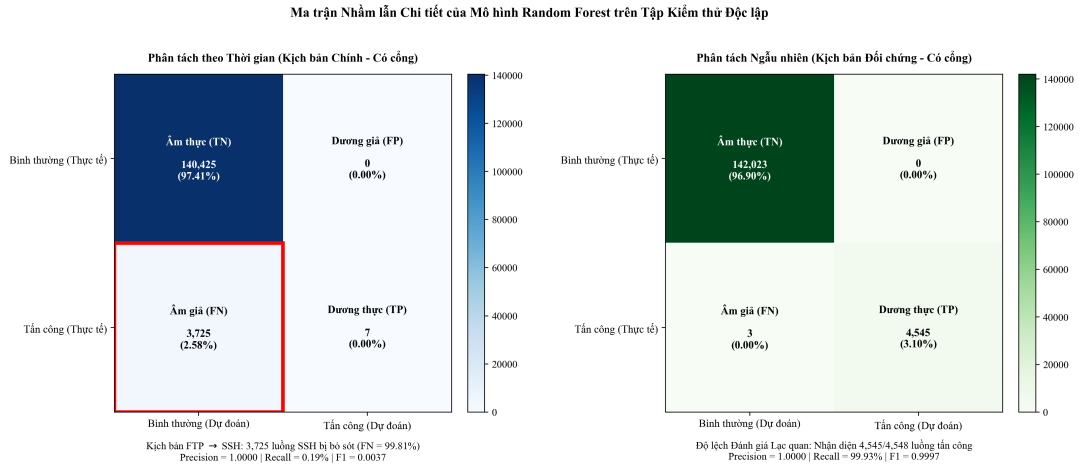
Bảng 6.2: Hiệu năng tổng thể trên tập Test đối chuẩn giữa 12 tổ hợp thực nghiệm (Đánh giá sau khi khóa cấu hình).

Mô hình	Phân tách	Kịch bản	Accuracy	Precision	Recall	F1	AP	ROC-AUC	FTP Rec.	SSH Rec.
A. Thực nghiệm chính: Phân tách theo thời gian (kịch bản FTP → SSH)										
Decision Tree	Thời gian	Có công	0.9740	0.0000	0.0000	0.0000	0.0259	0.5000	N/A	0.00%
Random Forest	Thời gian	Có công	0.9742	1.0000	0.0019	0.0037	0.3591	0.7483	N/A	0.19%
XGBoost	Thời gian	Có công	0.9741	0.0000	0.0000	0.0000	0.0879	0.7503	N/A	0.00%
Decision Tree	Thời gian	Không công	0.9741	0.4375	0.0019	0.0037	0.0272	0.5009	N/A	0.19%
Random Forest	Thời gian	Không công	0.9739	0.2414	0.0038	0.0074	0.0268	0.4932	N/A	0.38%
XGBoost	Thời gian	Không công	0.9740	0.2653	0.0035	0.0069	0.2447	0.9304	N/A	0.35%
B. Thực nghiệm đối chứng: Phân tách ngẫu nhiên (đối chứng rò rỉ dữ liệu)										
Decision Tree	Ngẫu nhiên	Có công	0.9996	0.9904	0.9980	0.9942	0.9963	0.9990	100.00%	99.54%
Random Forest	Ngẫu nhiên	Có công	1.0000	1.0000	0.9993	0.9997	1.0000	1.0000	100.00%	99.85%
XGBoost	Ngẫu nhiên	Có công	1.0000	0.9993	1.0000	0.9997	1.0000	1.0000	100.00%	100.00%
Decision Tree	Ngẫu nhiên	Không công	0.9989	0.9700	0.9949	0.9823	0.9890	0.9974	99.69%	99.23%
Random Forest	Ngẫu nhiên	Không công	0.9997	0.9934	0.9954	0.9944	0.9994	1.0000	99.73%	99.28%
XGBoost	Ngẫu nhiên	Không công	0.9999	0.9980	0.9987	0.9984	0.9999	1.0000	99.92%	99.79%

Kết quả thực nghiệm ghi nhận sự suy giảm hiệu năng rõ rệt đối với biến thể tấn công mới:

- **Cây Quyết Định (Decision Tree):** Đạt F1-Score = **0.0000**, Precision = 0.0000, Recall = 0.0000, Average Precision = 0.0259, ROC-AUC = 0.5000. Ma trận nhầm lẫn: $TP = 0$, $FP = 11$, $FN = 3,732$, $TN = 140,414$. Mô hình không nhận diện được mẫu tấn công SSH nào (SSH Recall = 0.00%).
- **Rừng Ngẫu Nhiên (Random Forest):** Đạt F1-Score = **0.0037**, Precision = 1.0000, Recall = 0.0019, Average Precision = 0.3591, ROC-AUC = 0.7483. Ma trận nhầm lẫn: $TP = 7$, $FP = 0$, $FN = 3,725$, $TN = 140,425$. Mô hình chỉ nhận diện chính xác **7 trên tổng số 3,732 flow tấn công SSH**, tương ứng với tỷ lệ phát hiện SSH Recall = **0.19%**.
- **XGBoost:** Đạt F1-Score = **0.0000**, Precision = 0.0000, Recall = 0.0000, Average Precision = 0.0879, ROC-AUC = 0.7503. Ma trận nhầm lẫn gồm $TP = 0$, $FP = 0$, $FN = 3,732$, $TN = 140,425$. Tương tự Decision Tree, XGBoost không nhận diện được cuộc tấn công SSH nào (SSH Recall = 0.00%).

Ma trận nhầm lẫn chi tiết của ba mô hình trên tập kiểm thử được trực quan hóa trên Hình 6.1.



Hình 6.1: Ma trận nhầm lẫn chi tiết của các mô hình trên tập kiểm thử phân tách theo thời gian, thể hiện trực quan tỷ lệ True Positive rất thấp đối với biến thể tấn công SSH.

Ghi nhận về Support và FTP Recall trên tập Test: Do tập $\mathcal{D}_{\text{test}}$ theo thời gian diễn ra từ 14:32 đến 17:00 chiều (chỉ xuất hiện chiến dịch SSH-Patator), số lượng mẫu hỗ trợ của FTP-Patator là 0 (FTP Support = 0). Theo đúng nguyên tắc trung thực học thuật đã đề ra, chỉ số **FTP Recall trên tập Test được ghi nhận là N/A (Not Applicable)**, không gán giá trị 0% hay 100%.

Nhận xét kết quả: Trong kịch bản đánh giá chính theo thời gian có công, Random Forest đạt F1-Score = 0.0037 với SSH Recall = 0.19% (7/3,732 flow SSH), trong khi Decision Tree và XGBoost có F1-Score = 0.0000. Kết quả này cung cấp bằng chứng thực nghiệm rõ ràng cho thấy: khi chỉ được huấn luyện trên tập Train chứa biến thể FTP-Patator, các mô hình học máy dạng cây gặp khó khăn lớn trong việc khái quát hóa sang biến thể SSH-Patator xuất hiện ở phân đoạn thời gian sau trên cùng hệ thống mạng. Mặc dù tập Validation có chứa 1,886 mẫu SSH tham gia lựa chọn mô hình, việc không có mẫu SSH trong quá trình fit khiến tỷ lệ phát hiện SSH trên tập Test ở mức rất thấp.

6.4 Kết quả thực nghiệm đối chứng trên phân tách ngẫu nhiên

Khác biệt rõ rệt với kết quả hiệu năng rất thấp trên phân tách theo thời gian, kết quả của các mô hình trên tập kiểm thử phân tách ngẫu nhiên (phân tách ngẫu nhiên có công) đạt điểm số rất cao (xem Phần B của Bảng 6.2):

- **Cây Quyết Định (Decision Tree):** Đạt F1-Score = **0.9942**, Precision = 0.9904, Recall = 0.9980, Average Precision = 0.9963, ROC-AUC = 0.9990. Tỷ lệ phát hiện đạt 100.00% đối với FTP và 99.54% đối với SSH.

- **Rừng Ngẫu Nhiên (Random Forest):** Đạt F1-Score = **0.9997**, Precision = 1.0000, Recall = 0.9993, Average Precision = 1.0000, ROC-AUC = 1.0000. Tỷ lệ phát hiện đạt 100.00% đối với FTP và 99.85% đối với SSH.
- **XGBoost:** Đạt F1-Score = **0.9997**, Precision = 0.9993, Recall = 1.0000, Average Precision = 1.0000, ROC-AUC = 1.0000. Tỷ lệ phát hiện đạt 100.00% đối với FTP và 100.00% đối với SSH.

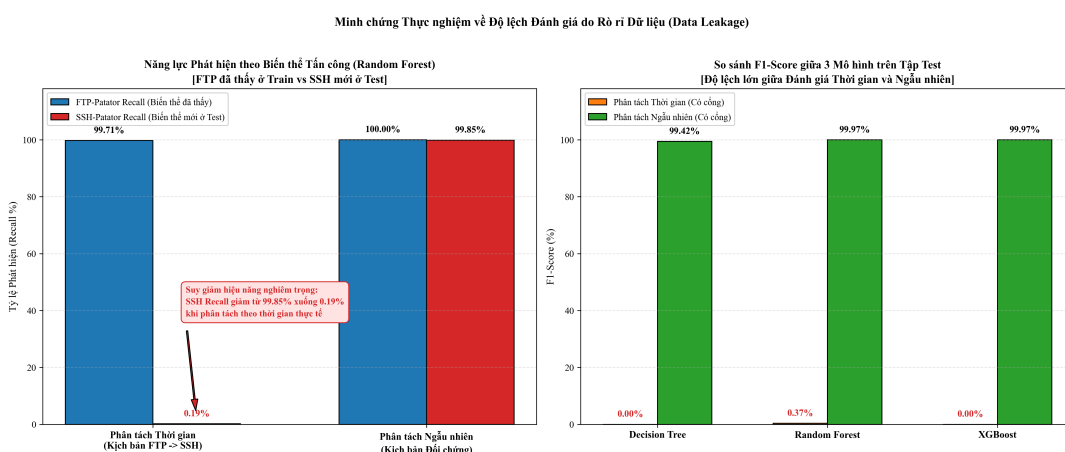
6.4.1 Đo lường định lượng độ lệch đánh giá

Bảng 6.3 trình bày sự chênh lệch trực tiếp giữa hai chiến lược phân tách dữ liệu trên tập kiểm định.

Bảng 6.3: Độ lệch hiệu năng ($\Delta F1$) giữa Phân tách Ngẫu nhiên và Phân tách Thời gian trên tập Validation.

Mô hình	Kịch bản công	F1 (Thời gian)	F1 (Ngẫu nhiên)	$\Delta F1$ (Random - Time)	SSH Recall (Random)
Decision Tree	Có công	0.7170	0.9939	0.2769	99.53%
Decision Tree	Không công	0.7169	0.9799	0.2631	98.87%
Random Forest	Có công	0.7181	0.9997	0.2816	99.89%
Random Forest	Không công	0.7134	0.9942	0.2808	99.05%
XGBoost	Có công	0.7181	0.9995	0.2814	100.00%
XGBoost	Không công	0.7136	0.9984	0.2848	99.67%

Khoảng cách chênh lệch hiệu năng ($\Delta F1$ -Score) giữa Random Split và Time-based Split trên tập Validation dao động từ **0.2631 đến 0.2848**. Trên tập Test chính thức, khoảng cách này thể hiện rất rõ: đối với Random Forest, điểm F1 tăng từ **0.0037** (trên Time Split) lên **0.9997** (trên Random Split), như được minh họa đối chiếu trên Hình 6.2.



Hình 6.2: Đối chiếu hiệu năng giữa phân tách ngẫu nhiên và phân tách theo thời gian, trực quan hóa mức độ chênh lệch hiệu năng do tương quan nội bộ chiến dịch.

Hiện tượng này phù hợp với các phân tích của Arp và cộng sự [4] cũng như Pendlebury và cộng sự [16]: Việc phân tách ngẫu nhiên trên dữ liệu chuỗi thời gian mạng có thể tạo ra độ

lệch đánh giá quá lạc quan (*optimistic evaluation bias*) vì các flow có tương quan mạnh từ cùng một chiến dịch tấn công (tương quan nội bộ chiến dịch – *within-campaign similarity*) được phân bổ vào cả tập huấn luyện và kiểm thử. Do đó, điểm số cao trong thiết lập ngẫu nhiên phản ánh một đánh giá mang tính chất nội suy (*interpolation-like evaluation*) trên các mẫu có độ trùng lặp phân phối cao, hơn là bằng chứng cho năng lực khái quát hóa sang các biến thể hoặc thời điểm mới.

6.5 Kết quả thực nghiệm bóc tách cổng mạng

Để kiểm chứng giả thuyết cho rằng mô hình gặp khó khăn trên tập Test theo thời gian chỉ đơn thuần là do dựa vào lỗi tắt cổng dịch vụ đích (`Destination Port`), nhóm tiến hành phân tích kịch bản bóc tách cổng (`without_port`), trong đó đặc trưng này bị loại bỏ khỏi không gian huấn luyện.

Bảng 6.4 tổng hợp kết quả bóc tách cổng trên tập kiểm định $\mathcal{D}_{\text{val}}$.

Bảng 6.4: Thực nghiệm Bóc tách Cổng mạng (Port Ablation) trên tập Validation phân tách theo thời gian.

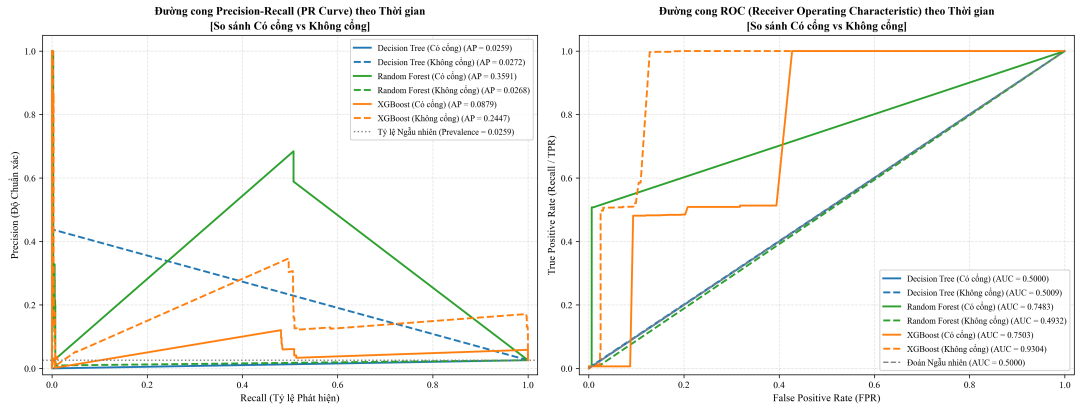
Mô hình	F1 (Có Cổng)	F1 (Không Cổng)	$\Delta F1$	SSH Recall (Có)	SSH Recall (Không)
Decision Tree	0.7170	0.7169	0.0001	0.00%	0.00%
Random Forest	0.7181	0.7134	0.0047	0.00%	0.00%
XGBoost	0.7181	0.7136	0.0045	0.00%	0.00%

Trên tập kiểm định $\mathcal{D}_{\text{val}}$, việc loại bỏ cổng làm giảm nhẹ điểm F1 của cả 3 mô hình (khoảng 0.0001 đến 0.0047), và tỷ lệ phát hiện tấn công SSH vẫn giữ ở mức 0.00% (do mô hình tối ưu theo mẫu FTP chiếm đa số trong tập Val).

Khi quan sát kết quả trên tập kiểm thử $\mathcal{D}_{\text{test}}$ theo thời gian (xem lại Phần A của Bảng 6.2):

- **Cây Quyết Định (Decision Tree):** Điểm F1 tăng nhẹ từ 0.0000 lên **0.0037**, nhận diện được 0.19% tấn công SSH (7 mẫu).
- **Rừng Ngẫu Nhiên (Random Forest):** Điểm F1 tăng từ 0.0037 lên **0.0074**, nhận diện được 0.38% tấn công SSH (14 trên tổng số 3,732 mẫu).
- **XGBoost:** Điểm F1 tăng từ 0.0000 lên **0.0069**, nhận diện được 0.35% tấn công SSH (13 trên tổng số 3,732 mẫu).

Sự thay đổi diện tích dưới đường cong ROC và đường cong Precision-Recall giữa hai kịch bản có và không có cổng được đối chiếu trực quan trên Hình 6.3.



Hình 6.3: Đường cong ROC và Precision-Recall đối chiếu giữa các mô hình và kịch bản thực nghiệm, phản ánh sự biến thiên diện tích dưới đường cong khi có và không có đặc trưng cổng mạng.

Kết luận về vai trò của cổng dịch vụ: Việc loại bỏ cổng dịch vụ giúp các mô hình nhận diện được thêm một lượng nhỏ các flow SSH dựa trên đặc trưng hành vi (tỷ lệ SSH Recall tăng từ 0.00% – 0.19% lên 0.19% – 0.38%).

Tuy nhiên, hiệu năng tổng thể của mô hình trong toàn bộ các tổ hợp theo thời gian vẫn ở mức rất thấp, với giá trị F1 cao nhất được ghi nhận là **0.0074** ở Random Forest (nhận diện được 0.38% tấn công SSH, tức 14/3,732 flow). Kết quả này cho thấy: mặc dù `Destination Port` là một lối tắt quan trọng của mô hình, việc loại bỏ cổng không đủ để giải quyết sự suy giảm hiệu năng tổng quát hóa. Nói cách khác, sự sụt giảm hiệu năng trong việc phát hiện SSH không thể được giải thích chỉ bằng riêng yếu tố cổng dịch vụ. Sự khác biệt về cơ chế hoạt động và phân phối đặc trưng thống kê gói tin giữa hai giao thức FTP và SSH (kết hợp với yếu tố thời gian và chiến dịch tấn công) là một cơ chế hợp lý góp phần tạo nên sự dịch chuyển phân phối này.

6.6 Phân tích khả năng giải thích

Để làm rõ hành vi dự đoán bên trong của mô hình trên tập mẫu kiểm định, nhóm áp dụng phương pháp giải thích gia tăng Shapley (**SHAP TreeExplainer** [10]) kết hợp với việc trích xuất cấu trúc logic của Cây Quyết Định.

6.6.1 Phân tích tầm quan trọng đặc trưng bằng SHAP

Phân tích SHAP được thực hiện trên 500 mẫu đại diện từ tập kiểm định $\mathcal{D}_{val}$ đối với mô hình Random Forest theo thời gian có cổng. Bảng 6.5 liệt kê 10 đặc trưng có giá trị đóng góp trung bình tuyệt đối ($\text{Mean } |\text{SHAP}|$) lớn nhất.

Bảng 6.5: Top 10 đặc trưng mạng có đóng góp lớn nhất vào quyết định phân loại theo giá trị SHAP trung bình.

Hạng	Tên đặc trưng mạng	Mean SHAP	Ý nghĩa giao thức / thống kê mạng
1	Destination Port	0.1237	Cổng đích dịch vụ (Port 21 FTP, Port 22 SSH)
2	Fwd Packet Length Std	0.0339	Độ lệch chuẩn kích thước gói tin hướng đi (Fwd)
3	Avg Fwd Segment Size	0.0292	Kích thước phân đoạn trung bình hướng đi
4	Max Packet Length	0.0289	Độ dài gói tin lớn nhất quan sát được trong flow
5	Packet Length Mean	0.0275	Độ dài gói tin trung bình của toàn flow
6	min_seg_size_forward	0.0272	Kích thước header TCP tối thiểu phía forward
7	Packet Length Std	0.0271	Độ lệch chuẩn độ dài toàn bộ gói tin trong flow
8	Fwd Packet Length Mean	0.0271	Độ dài gói tin trung bình hướng đi (Fwd)
9	Average Packet Size	0.0262	Kích thước gói tin trung bình tổng thể
10	Packet Length Variance	0.0208	Phương sai phân bố độ dài gói tin trong flow

Kết quả định lượng cho thấy:

- Đặc trưng **Destination Port** có giá trị lớn nhất với Mean |SHAP| = **0.1237**.
- Đặc trưng xếp thứ hai là Fwd Packet Length Std đạt giá trị **0.0339**.
- Tỷ số đóng góp giữa đặc trưng số 1 và số 2 là xấp xỉ **3.6 lần**.

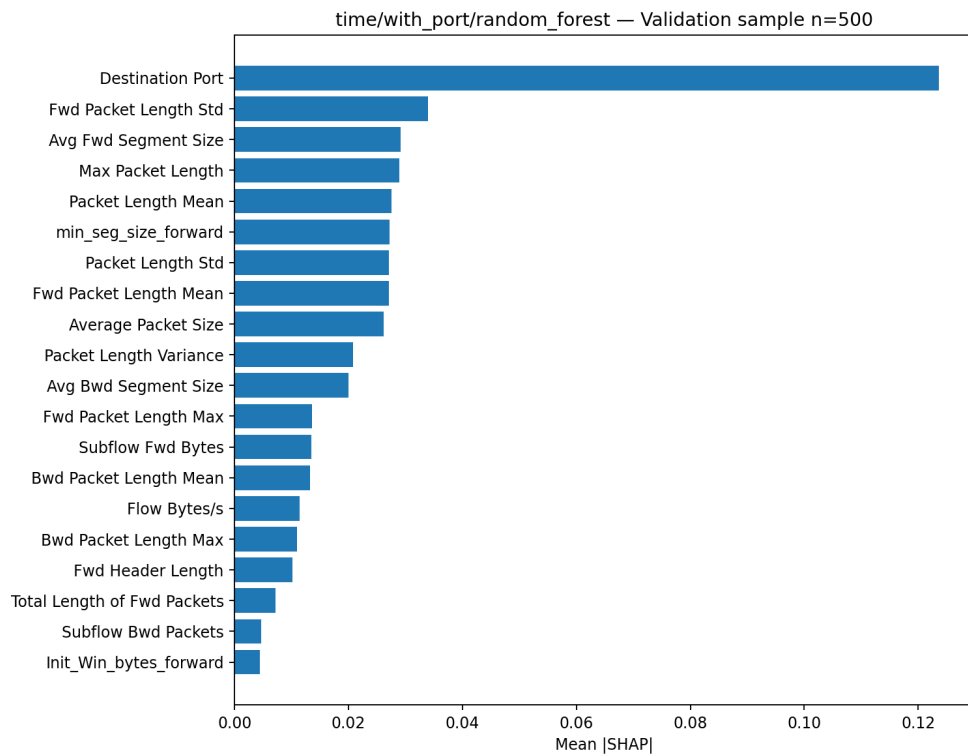
Kết quả phân tích SHAP cho thấy mô hình phụ thuộc mạnh vào Destination Port trong kịch bản có cổng được phân tích, tận dụng cổng dịch vụ như một lối tắt phân loại quan trọng giữa lưu lượng bình thường và lưu lượng tấn công FTP (Cổng 21), như được minh họa trực quan trên biểu đồ phân bố giá trị SHAP (Hình 6.4).

6.6.2 Trích xuất cấu trúc cây quyết định

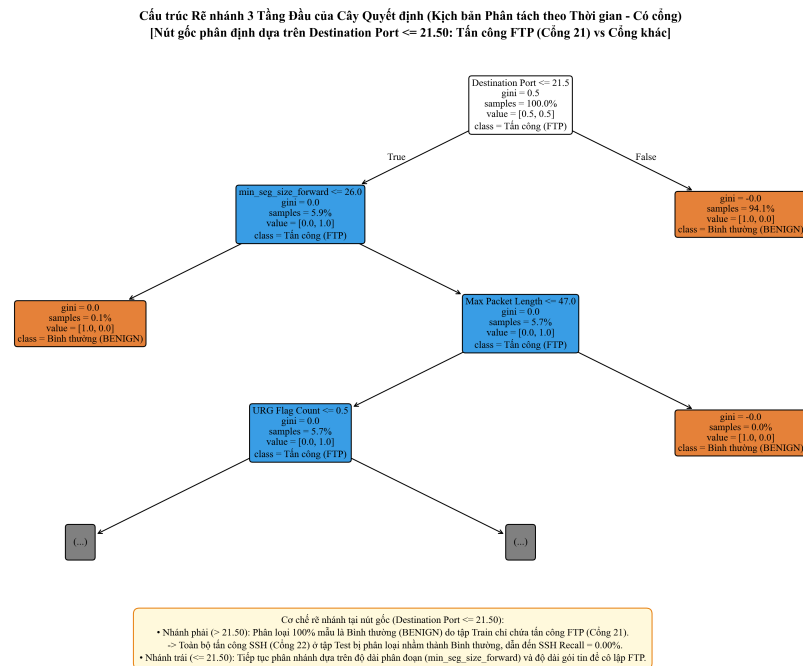
Quan sát cấu trúc logic của Cây Quyết Định càng làm sáng tỏ cơ chế phân nhánh này:

- **Kịch bản có cổng:** Nút gốc (Root Node) phân chia dựa trên điều kiện Destination Port ≤ 21.5 (Hình 6.5). Các nhánh tiếp theo tiếp tục cô lập các flow có Destination Port ≤ 20.5 (loại bỏ traffic cổng thấp) để định vị chính xác **Cổng 21 (FTP)**.
- **Kịch bản không có cổng:** Khi đặc trưng cổng bị loại bỏ, cây quyết định chọn nút gốc là Average Packet Size ≤ 14.12 (chi tiết quy tắc văn bản trình bày tại Phụ lục D).

Khi chuyển sang tập kiểm thử $\mathcal{D}_{\text{test}}$, toàn bộ các cuộc tấn công đều diễn ra trên **Cổng 22 (SSH)**, tức là rơi vào nhánh Destination Port > 21.5 . Tại nhánh này, vì trong tập Train mô hình chưa từng thấy cuộc tấn công nào diễn ra trên cổng > 21 , phần lớn các nút lá



Hình 6.4: Biểu đồ phân bố giá trị SHAP (SHAP Summary Plot) của Top các đặc trưng quan trọng trên 500 mẫu kiểm định.



Hình 6.5: Cấu trúc 3 tầng đầu tiên của Cây Quyết Định minh họa ngưỡng rẽ nhánh ưu tiên tại Destination Port ≤ 21.5 .

tương ứng đều được gán nhãn là lưu lượng bình thường (Normal, 0). Cấu trúc nhánh rẽ này giải thích vì sao cây quyết định không phát hiện được các flow SSH ở Cổng 22 trong kịch bản có cổng. Cần nhấn mạnh rằng SHAP và cấu trúc cây giải thích hành vi phân loại của mô hình trên tập mẫu quan sát, chứ không chứng minh quan hệ nhân quả trong môi trường mạng thực tế.

6.7 Nghiên cứu điển hình: Tác động của thiết kế pipeline và cờ refit trên XGBoost

Trong quá trình đối soát thực nghiệm giữa các đường ống trong mã nguồn, nhóm phát hiện sự phân kỳ số liệu đáng chú ý đối với thuật toán XGBoost:

- Trong benchmark chuẩn mực tại `artifacts/week3_week4/test_comparison.csv`, XGBoost đạt F1-Score = **0.0000** (SSH Recall = 0.00%).
- Trong kết quả tuning độc lập tại `experiments/w3_05_time_tuning/test_eval/test_metrics.json`, XGBoost lại đạt F1-Score = **0.9912** (SSH Recall = **98.34%**).

6.7.1 Truy vết nguyên nhân kỹ thuật trong mã nguồn

Tiến hành phân tích mã nguồn `src/ids/tune_xgboost.py` (khởi điều phối tuning và refit), cơ chế gây phân kỳ số liệu được thể hiện qua cờ `strict_train_only`:

Listing 6.1: Đoạn mã cấu hình RandomizedSearchCV và rẽ nhánh fit mô hình trong `src/ids/tune_xgboost.py`.

```
search = RandomizedSearchCV(
    estimator=base_model,
    param_distributions=param_distributions,
    n_iter=n_iter,
    scoring=f1_scorer,
    cv=cv_split,
    refit=not strict_train_only, # Mặc định False -> refit=True
    verbose=2,
    random_state=random_state,
    n_jobs=1,
    return_train_score=True
)

# Gộp tập Train và Validation để chạy phân tách tùy biến cv_split
X_combined = pd.concat([X_train, X_val], axis=0).reset_index(drop=True)
y_combined = pd.concat([y_train, y_val], axis=0).reset_index(drop=True)
search.fit(X_combined, y_combined)
```

```

if strict_train_only:
    # Che do chuan: fit lai chi tren duy nhat X_train
    best_model = clone(base_model).set_params(**search.best_params_)
    best_model.fit(X_train, y_train)
else:
    # Che do mac dinh CLI: lay truc tiep best_estimator_ da refit tren X_combined
    best_model = search.best_estimator_

joblib.dump(best_model, best_model_path)

```

Theo tài liệu đặc tả của thư viện `scikit-learn` [15], khi tham số `refit=True` (tương ứng với chế độ mặc định của CLI khi không truyền cờ `--strict-train-only`), sau khi hoàn tất việc đánh giá các tổ hợp siêu tham số trên `cv_split` (sử dụng tập con Train làm fold huấn luyện và Val làm fold kiểm định), đối tượng `RandomizedSearchCV` hoạt động đúng theo đặc tả: gọi hàm `fit()` của mô hình tốt nhất trên toàn bộ tập dữ liệu đầu vào đã truyền vào hàm `fit()`, tức là toàn bộ tập gộp $\mathbf{X}_{\text{combined}} = \mathbf{X}_{\text{train}} \cup \mathbf{X}_{\text{val}}$.

Nguyên nhân dẫn đến phân kỳ số liệu nằm ở thiết kế đường ống: dữ liệu Train và Validation được chủ động gộp thành $\mathbf{X}_{\text{combined}}$ trước khi truyền vào `RandomizedSearchCV.fit()`, trong khi chế độ mặc định `refit=True` khiến mô hình được huấn luyện lại trên toàn bộ tập gộp đó thay vì chỉ trên $\mathbf{X}_{\text{train}}$. Khi truyền cờ `--strict-train-only`, script chủ động clone cấu hình tối ưu và fit lại duy nhất trên $\mathbf{X}_{\text{train}}$, bảo toàn bản chất tổng quát hóa sang subtype chưa từng thấy.

6.7.2 Hậu quả đối với bản chất bài toán đánh giá

Trong khi tập $\mathcal{D}_{\text{train}}$ không chứa mẫu SSH-Patator nào (0 flow), thì tập kiểm định $\mathcal{D}_{\text{val}}$ lại chứa **1,886 flow tấn công SSH-Patator**.

Khi cơ chế `refit=True` diễn ra trên tập dữ liệu gộp:

1. Mô hình XGBoost được huấn luyện trực tiếp trên 1,886 mẫu tấn công SSH từ tập Validation.
2. Bài toán đánh giá trên tập Test theo thời gian đã bị biến đổi bản chất từ **kịch bản tổng quát hóa FTP $\rightarrow$ SSH (không có SSH trong tập huấn luyện)** thành **học có giám sát trên biến thể đã biết (Seen-Subtype Supervised Learning)** do được huấn luyện trên **1,886 mẫu SSH**.
3. Việc được học trên 1,886 mẫu SSH này đã giúp XGBoost tiếp thu phân phối đặc trưng và công 22 của SSH, làm cho điểm số trên tập Test tăng từ **0.0000 lên 0.9912**.

Bảng 6.6 tổng hợp sự đối lập giữa hai chế độ đánh giá này.

Bảng 6.6: Đối chiếu hiệu năng của XGBoost giữa chế độ đánh giá tổng quát hóa chuẩn mực và chế độ tiếp xúc validation do thiết kế pipeline (Refit).

Chế độ thực nghiệm	Dữ liệu thực tế fit mô hình	Mẫu SSH trong tập fit	Test F1-Score	Test SSH Recall
Chế độ 1: Tổng quát hóa chuẩn mực	Duy nhất Train ($\mathcal{D}_{\text{train}}$)	0 mẫu	0.0000	0.00%
Chế độ 2: Pipeline gộp tập (Refit)	Train + Val ($\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}}$)	1,886 mẫu	0.9912	98.34%

Phát hiện này mang lại một bài học thực hành giá trị trong kỹ nghệ MLOps: việc chủ động gộp Train và Validation trước khi gọi hàm tìm kiếm siêu tham số kết hợp với hành vi mặc định `refit=True` của thư viện có thể làm thay đổi bản chất bài toán đánh giá nếu không được kiểm soát ranh giới dữ liệu một cách cẩn trọng.

Chương 7

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết toàn bộ các kết quả đạt được từ quá trình nghiên cứu thực nghiệm, trả lời trực tiếp bốn câu hỏi nghiên cứu đã đặt ra, đồng thời phân tích các mối đe dọa đến tính hợp lệ, đề xuất các hướng mở rộng trong tương lai và báo cáo chi tiết bảng phân công nhiệm vụ của các thành viên trong nhóm.

7.1 Tổng kết kết quả và trả lời các câu hỏi nghiên cứu

Qua quá trình thực nghiệm nghiêm ngặt trên tập dữ liệu CICIDS2017 với 12 tổ hợp mô hình và kịch bản dữ liệu, nhóm nghiên cứu rút ra các kết luận định lượng cụ thể nhằm trả lời bốn câu hỏi nghiên cứu:

1. Câu hỏi nghiên cứu 1 (RQ1 – Khả năng tổng quát hóa sang biến thể mới theo thời gian):

Khi mô hình học máy dạng cây chỉ được huấn luyện trên tập Train chứa biến thể FTP-Patator, khả năng phát hiện biến thể SSH-Patator xuất hiện ở giai đoạn thời gian sau suy giảm nghiêm trọng. Trong kịch bản thực nghiệm chính có cổng mạng (time/with_port), Rừng ngẫu nhiên chỉ đạt F1-Score = 0.0037 với SSH Recall = 0.19% (chỉ nhận diện chính xác 7 trên tổng số 3,732 luồng tấn công SSH), trong khi Cây quyết định và XGBoost hoàn toàn không phát hiện được bất kỳ luồng SSH nào (F1-Score = 0.0000, SSH Recall = 0.00%). Mặc dù tập kiểm định có chứa 1,886 mẫu SSH tham gia vào quá trình lựa chọn siêu tham số, việc mô hình không được huấn luyện trên mẫu SSH nào trong tập Train vẫn khiến tỷ lệ phát hiện biến thể mới trên tập kiểm thử ở mức rất thấp.

2. Câu hỏi nghiên cứu 2 (RQ2 – Mức độ ảnh hưởng của phép phân tách ngẫu nhiên):

Phép phân tách ngẫu nhiên (Random Split) tạo ra sự chênh lệch hiệu năng rất lớn so với phép phân tách theo thời gian (Time-based Split). Trên tập kiểm thử, điểm F1 của Rừng ngẫu nhiên tăng vọt từ 0.0037 (theo thời gian) lên 0.9997 (ngẫu nhiên); tỷ lệ phát hiện SSH tăng từ 0.19% lên 99.85%. Kết quả này khẳng định: việc phân tách ngẫu nhiên trên dữ liệu lưu lượng mạng theo chuỗi thời gian gây ra độ lệch đánh giá quá lạc quan (*optimistic evaluation bias*). Nguyên nhân cốt lõi là do sự tương quan mạnh

giữa các luồng trong cùng một chiến dịch tấn công (*within-campaign similarity*) bị rò rỉ đồng thời vào cả tập huấn luyện và tập kiểm thử, biến bài toán kiểm thử thành một đánh giá mang tính chất nội suy (*interpolation-like evaluation*) thay vì phản ánh năng lực khái quát hóa sang các biến thể hoặc mốc thời gian mới.

3. Câu hỏi nghiên cứu 3 (RQ3 – Vai trò của cổng dịch vụ và các đặc trưng hành vi):

Cổng dịch vụ đích (`Destination Port`) đóng vai trò là một lối tắt phân loại (*shortcut feature*) chi phối mạnh mẽ quyết định của mô hình. Phân tích SHAP chỉ ra cổng dịch vụ có giá trị đóng góp trung bình tuyệt đối $\text{Mean } |\text{SHAP}| = 0.1237$, gấp 3.6 lần đặc trưng xếp thứ hai. Khi loại bỏ cổng dịch vụ khỏi không gian huấn luyện (`without_port`), mô hình nhận diện được thêm một lượng nhỏ các luồng SSH dựa trên hình thái thống kê gói tin (tỷ lệ SSH Recall tăng lên mức 0.19%–0.38%), với F1 cao nhất đạt 0.0074 ở Rừng ngẫu nhiên (14/3,732 luồng SSH). Tuy nhiên, hiệu năng tổng thể vẫn ở mức rất thấp, chứng minh rằng sự thất bại trong việc phát hiện SSH không thể được giải thích chỉ bằng riêng yếu tố cổng dịch vụ. Sự khác biệt về cơ chế hoạt động và phân phối đặc trưng hình thái gói tin giữa hai giao thức FTP và SSH kết hợp với yếu tố thời gian là nguyên nhân sâu xa dẫn đến sự dịch chuyển phân phối dữ liệu quan sát được.

4. Câu hỏi nghiên cứu 4 (RQ4 – Tác động của thiết kế pipeline và cờ huấn luyện lại):

Lỗi thiết kế quy trình khi chủ động gộp tập Train và Validation trước khi đưa vào tìm kiếm siêu tham số, kết hợp với hành vi mặc định `refit=True` của thư viện scikit-learn [15], đã vô tình huấn luyện lại mô hình trên toàn bộ dữ liệu gộp chứa 1,886 mẫu SSH từ tập kiểm định. Điều này đã làm thay đổi hoàn toàn bản chất bài toán: từ đánh giá tổng quát hóa sang biến thể mới thành bài toán học có giám sát trên biến thể đã biết, khiến điểm F1 trên tập kiểm thử tăng vọt từ 0.0000 lên 0.9912 (nhận diện được 98.34% luồng SSH). Nghiên cứu này là minh chứng thực nghiệm sâu sắc về tầm quan trọng của việc kiểm soát ranh giới dữ liệu trong kỹ thuật vận hành học máy (MLOps).

7.2 Đánh giá tính hợp lệ và các giới hạn của đề tài

Bảng 7.1 tổng hợp các mối đe dọa đến tính hợp lệ của nghiên cứu và các biện pháp kiểm soát khoa học tương ứng đã được nhóm áp dụng xuyên suốt đề tài.

Bên cạnh các kết quả đạt được, nhóm nghiên cứu cũng thẳng thắn ghi nhận các giới hạn phạm vi của bài tập lớn:

- **Phạm vi dữ liệu:** Toàn bộ thực nghiệm được thực hiện trên tập dữ liệu mô phỏng CICIDS2017 Tuesday (gồm lưu lượng bình thường và hai chiến dịch tấn công Brute Force FTP và SSH). Do đó, các kết luận phản ánh đặc tính của bộ dữ liệu thực nghiệm

Bảng 7.1: Tổng hợp các mối đe dọa đến tính hợp lệ của nghiên cứu và giải pháp kiểm soát.

Khía cạnh hợp lệ	Nguy cơ tiềm ẩn	Biện pháp kiểm soát trong nghiên cứu
Tính hợp lệ bên trong (<i>Internal Validity</i>)	Sai lệch do phục hồi thời gian hoặc rò rỉ thông tin ranh giới giữa các tập dữ liệu.	Áp dụng Purge ($q = 60s$) và Embargo ($g = 120s$); loại bỏ 7,632 dòng ranh giới; kiểm tra toàn vẹn chuỗi băm SHA-256.
Tính hợp lệ xây dựng (<i>Construct Validity</i>)	Nhầm lẫn giữa phân loại luồng đã hoàn tất và hệ thống NIDS thời gian thực.	Tuyên bố minh bạch phạm vi phân loại luồng đã hoàn tất (<i>Completed-Flow Classification</i>); không đo lường độ trễ phát hiện gói tin; không tuyên bố triển khai thực tuyến.
Tính hợp lệ bên ngoài (<i>External Validity</i>)	Dữ liệu mô phỏng trong môi trường phòng thí nghiệm (CICIDS2017).	Phân tích cơ chế suy giảm tổng quát hóa; nhấn mạnh yêu cầu đánh giá trên lưu lượng mạng đa dạng và hạ tầng thực tế.
Tính hợp lệ thống kê (<i>Statistical Validity</i>)	Tỷ lệ tấn công thấp (2%–5%) gây nhiễu chỉ số Accuracy.	Sử dụng F1-Score, đường cong Precision-Recall, AP và FPR làm hệ quy chiếu cốt lõi; loại bỏ sự phụ thuộc vào nghịch lý độ chính xác (<i>Accuracy Paradox</i>).

cụ thể và cần được kiểm chứng thêm trước khi khái quát hóa cho mọi môi trường mạng thực tế.

- **Cơ chế phân loại luồng đã đóng:** Hệ thống hoạt động theo cơ chế phân loại luồng đã hoàn tất (*Completed-Flow Classification*) dựa trên các đặc trưng thống kê tổng hợp của công cụ CICFlowMeter [5]. Hệ thống không thực hiện đo lường độ trễ xử lý gói tin ở mức thời gian thực và không thay thế trực tiếp cho các hệ thống ngăn ngừa xâm nhập mạng (IPS) chặn gói tin thực tuyến như Snort [1] hay Suricata [2].
- **Tính tái lập:** Toàn bộ mã nguồn, cấu hình siêu tham số và mã băm SHA-256 đã được khóa và lưu vết độc lập trong tệp `frozen.json`, đảm bảo khả năng tái lập kết quả trong cùng điều kiện môi trường thực nghiệm.

7.3 Hướng phát triển và mở rộng

Từ các phát hiện thực nghiệm và giới hạn đã xác định, nhóm đề xuất một số hướng phát triển tiếp theo cho đề tài:

1. **Mở rộng không gian tấn công Brute Force:** Đánh giá năng lực tổng quát hóa của mô hình trên các giao thức dò quét mật khẩu phổ biến khác như RDP (Cổng 3389), SMB (Cổng 445), Telnet (Cổng 23) và xác thực HTTP POST.
2. **Nghiên cứu cơ chế phân loại luồng sớm (Early Flow Classification):** Thay vì đợi luồng mạng kết thúc hoàn toàn (vốn có thể kéo dài hàng chục giây), nghiên cứu trích

xuất đặc trưng thống kê chỉ từ N gói tin đầu tiên ($N \in \{5, 10, 20\}$) của mỗi phiên kết nối nhằm giảm thiểu tối đa độ trễ phát hiện trong thực tế.

3. **Mô hình kết hợp (Hybrid Architecture):** Kết hợp giữa hệ thống luật chuyên gia dựa trên chữ ký (Signature-based IDS) để xử lý các thuộc tính cổng/giao thức tĩnh, kết hợp với các mô hình học máy phi giám sát hoặc phát hiện bất thường (Anomaly Detection) để nhận diện các hành vi bất thường trong phân phối thời gian và độ dài gói tin.
4. **Thực nghiệm trên lưu lượng mạng thực tế:** Thu thập và kiểm thử mô hình trên dữ liệu lưu lượng mạng thực tế tại trung tâm dữ liệu hoặc mạng nội bộ trường đại học để đánh giá mức độ thích ứng trước các biến động lưu lượng thực tế.

7.4 Bảng phân công nhiệm vụ và đóng góp của thành viên

Để đảm bảo tính minh bạch, công bằng và hiệu quả trong quá trình thực hiện bài tập lớn, nhóm sinh viên đã phân chia nhiệm vụ chuyên trách theo từng mảng công việc cụ thể. Bảng 7.2 trình bày chi tiết trách nhiệm và tỷ lệ đóng góp của từng thành viên trong nhóm.

Bảng 7.2: Bảng phân công nhiệm vụ và tỷ lệ đóng góp của các thành viên trong nhóm.

STT	Họ và tên	Mã số SV	Nhiệm vụ chuyên trách	Đóng góp
1	Nguyễn Việt Tuấn Anh	0204068	Nghiên cứu cơ sở lý thuyết NIDS; khảo sát giao thức FTP/SSH và tấn công Patator [17]; xây dựng pipeline tiền xử lý dữ liệu và làm sạch ranh giới Purge/Embargo.	20%
2	Trần Phùng Đức Anh	0204168	Thiết kế quy trình phân tách dữ liệu theo thời gian và ngẫu nhiên; huấn luyện, tinh chỉnh siêu tham số mô hình Cây quyết định và Rừng ngẫu nhiên; tổng hợp báo cáo và đồ thị thực nghiệm.	20%
3	Nguyễn Vũ Dũng	0205968	Cấu hình và thực thi mô hình XGBoost; triển khai thực nghiệm bóc tách cổng mạng (Port Ablation); phân tích thực nghiệm trường hợp điển hình về thiết kế pipeline và cờ <code>refit=True</code> .	20%
4	Nguyễn Thu Hương	0209168	Triển khai phân tích khả năng giải thích bằng SHAP TreeExplainer [10]; trích xuất và đối chiếu cấu trúc rẽ nhánh cây quyết định; xây dựng bảng biểu thống kê đặc trưng.	20%
5	Võ Nguyễn Anh Kiệt	0209968	Xây dựng hệ thống kiểm thử tự động; quản lý và kiểm soát toàn vẹn chuỗi băm SHA-256 (<code>frozen.json</code>); rà soát các mối đe dọa đến tính hợp lệ và biên tập tài liệu kỹ thuật LaTeX.	20%

TÀI LIỆU THAM KHẢO

- [1] M. Roesch, “Snort: Lightweight intrusion detection for networks,” in *Proceedings of the 13th USENIX Conference on System Administration (LISA 99)*, 1999, pp. 229–238.
- [2] Open Information Security Foundation (OISF). “Suricata: Open source network analysis and threat detection engine,” Accessed: Sep. 17, 2026. [Online]. Available: <https://suricata.io/>
- [3] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *2010 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2010, pp. 305–316. DOI: [10.1109/SP.2010.25](https://doi.org/10.1109/SP.2010.25)
- [4] D. Arp et al., “Dos and don’ts of machine learning in computer security,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3971–3988. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>
- [5] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116. DOI: [10.5220/0006639801080116](https://doi.org/10.5220/0006639801080116)
- [6] B. Claise, B. Trammell, and P. Aitken, “Specification of the IP flow information export (IPFIX) protocol for the exchange of flow information,” RFC 7011, Sep. 2013. DOI: [10.17487/RFC7011](https://doi.org/10.17487/RFC7011) [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7011>
- [7] J. R. Quinlan, “Induction of decision trees,” *Machine learning*, vol. 1, no. 1, pp. 81–106, 1986. DOI: [10.1007/BF00116251](https://doi.org/10.1007/BF00116251)
- [8] L. Breiman, “Random forests,” *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324)
- [9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785)

- [10] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765–4774. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>
- [11] J. Davis and M. Goadrich, “The relationship between Precision-Recall and ROC curves,” in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 2006, pp. 233–240. DOI: 10.1145/1143844.1143874
- [12] scikit-learn developers. “Scikit-learn: Average_precision_score metric documentation,” Accessed: Sep. 17, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html
- [13] Canadian Institute for Cybersecurity. “CICIDS2017 dataset,” University of New Brunswick, Accessed: Sep. 17, 2026. [Online]. Available: <https://www.unb.ca/cic/datasets/ids-2017.html>
- [14] R. Geirhos et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020. DOI: 10.1038/s42256-020-00257-z
- [15] scikit-learn developers. “Scikit-learn: RandomizedSearchCV model evaluation parameter documentation,” Accessed: Sep. 17, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html
- [16] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, “TESSERACT: Eliminating experimental bias in malware classification across space and time,” in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 729–746. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury>
- [17] J.-B. Lorenzo. “Patator: Multi-purpose brute-forcer,” Accessed: Sep. 17, 2026. [Online]. Available: <https://github.com/lanjelot/patator>

Phụ lục A

HƯỚNG DẪN CÀI ĐẶT VÀ TÁI LẬP THỰC NGHIỆM

A.1 Yêu cầu môi trường và cài đặt phụ thuộc

Dự án hỗ trợ môi trường Python trong dải tương thích $\geq 3.11, < 3.13$ (khai báo tại `pyproject.toml`). Môi trường máy trạm thực tế dùng để chạy và kiểm chứng toàn bộ số liệu báo cáo là Python 3.12 trên Windows 11 với các thư viện đã được xác thực: `numpy==1.26.4`, `pandas==2.2.2`, `scikit-learn==1.6.1`, `xgboost==3.0.5`, `shap==0.48.0` và `matplotlib==3.9.2`.

Để cài đặt mã nguồn cùng các thành phần giải thích mô hình (SHAP/XAI), người nghiên cứu thực hiện các bước sau:

Listing A.1: Thiết lập môi trường ảo và cài đặt thư viện bao gồm gói mở rộng XAI

```
# 1. Clone repository
git clone https://github.com/Duc-AnhTp/cicids-bruteforce-ids.git
cd cicids-bruteforce-ids

# 2. Tạo và kích hoạt môi trường ảo (khuyến nghị Python 3.12)
python -m venv .venv
# Trên Linux / macOS:
source .venv/bin/activate
# Trên Windows PowerShell:
.\.venv\Scripts\Activate.ps1

# 3. Cài đặt package o che do editable kèm gói mở rộng giải thích SHAP (xai)
pip install -e ".[xai]"
```

A.2 Chuẩn bị dữ liệu và phân tách

Listing A.2: Thực hiện làm sạch và phân tách dữ liệu chuẩn mực

```
# Kiểm tra mã băm SHA-256 của tệp CSV gốc (Linux / macOS: sha256sum, Windows:
certutil)
```

```
sha256sum data/raw/Tuesday-WorkingHours.pcap_ISCX.csv
# hoặc chạy kiểm toàn toàn vẹn và phân bố thời gian bằng CLI của hệ thống:
python -m ids audit

# Chạy quy trình phục hồi timestamp và chia tách dữ liệu theo thời gian
python scripts/prepare_splits.py

# Tiến xử lý Train-only và loại bỏ 10 đặc trưng hàng số
python scripts/preprocess_data.py
```

A.3 Chạy thực nghiệm và kiểm thử tự động

Listing A.3: Huấn luyện, tìm kiếm siêu tham số và chạy bộ kiểm thử chuẩn CI

```
# Chạy toàn bộ quy trình benchmark (đóng băng và đánh giá Test)
python scripts/run_remaining.py

# Sinh lại toàn bộ macro số liệu và bảng LaTeX từ artifact
python scripts/report/generate_latex_registry.py

# Chạy bộ kiểm thử tự động (Unit & Protocol tests khớp cấu hình CI)
python -m unittest discover -s tests -v

# Chạy smoke test kiểm tra quy trình và giải thích mô hình
python scripts/smoke_test.py --output ci_demo --explain
```


Phụ lục B

KHÔNG GIAN SIÊU THAM SỐ VÀ CẤU HÌNH TỐI ƯU

Phụ lục này cung cấp chi tiết toàn bộ các cấu hình siêu tham số tối ưu của 12 tổ hợp thực nghiệm được lựa chọn từ tập lưu vết cấu hình đóng băng `frozen.json` và `validation_winners.csv`:

Bảng B.1: Chi tiết tham số tối ưu của 12 mô hình được lựa chọn trên tập Validation.

Mã kịch bản	Mô hình	Phân tách	Cổng	Tham số tối ưu được khóa
TN-01	Decision Tree	Thời gian	Có cổng	<code>criterion='gini', max_depth=6, min_samples_leaf=5</code>
TN-02	Random Forest	Thời gian	Có cổng	<code>n_estimators=100, max_depth=16, min_samples_leaf=5</code>
TN-03	XGBoost	Thời gian	Có cổng	<code>n_estimators=200, max_depth=4, learning_rate=0.08</code>
TN-04	Decision Tree	Thời gian	Không cổng	<code>criterion='gini', max_depth=8, min_samples_leaf=5</code>
TN-05	Random Forest	Thời gian	Không cổng	<code>n_estimators=100, max_depth=10, min_samples_leaf=5</code>
TN-06	XGBoost	Thời gian	Không cổng	<code>n_estimators=200, max_depth=4, learning_rate=0.08</code>
TN-07	Decision Tree	Ngẫu nhiên	Có cổng	<code>criterion='gini', max_depth=12, min_samples_leaf=5</code>
TN-08	Random Forest	Ngẫu nhiên	Có cổng	<code>n_estimators=100, max_depth=None, min_samples_leaf=5</code>
TN-09	XGBoost	Ngẫu nhiên	Có cổng	<code>n_estimators=200, max_depth=6, learning_rate=0.08</code>
TN-10	Decision Tree	Ngẫu nhiên	Không cổng	<code>criterion='gini', max_depth=None, min_samples_leaf=5</code>
TN-11	Random Forest	Ngẫu nhiên	Không cổng	<code>n_estimators=150, max_depth=None, min_samples_leaf=5</code>
TN-12	XGBoost	Ngẫu nhiên	Không cổng	<code>n_estimators=200, max_depth=6, learning_rate=0.08</code>

Phụ lục C

DANH MỤC ĐẦY ĐỦ CÁC ĐẶC TRƯNG MẠNG

Bảng sau liệt kê đầy đủ 67 đặc trưng mạng được sử dụng trong kịch bản `with_port` (trong kịch bản `without_port`, đặc trưng số 1 `Destination Port` bị loại bỏ):

STT	Tên đặc trưng trong dữ liệu	Ý nghĩa giao thức và thống kê luồng mạng
1	<code>Destination Port</code>	Cổng dịch vụ mạng đích (21 FTP, 22 SSH, 80 HTTP, 443 HTTPS)
2	<code>Flow Duration</code>	Tổng thời lượng tồn tại của phiên truyền thông luồng (μs)
3	<code>Total Fwd Packets</code>	Tổng số lượng gói tin truyền theo hướng đi (Client $\rightarrow$ Server)
4	<code>Total Backward Packets</code>	Tổng số lượng gói tin truyền theo hướng về (Server $\rightarrow$ Client)
5	<code>Total Length of Fwd Packets</code>	Tổng kích thước dữ liệu gói tin truyền theo hướng đi (bytes)
6	<code>Total Length of Bwd Packets</code>	Tổng kích thước dữ liệu gói tin truyền theo hướng về (bytes)
7	<code>Fwd Packet Length Max</code>	Kích thước gói tin lớn nhất theo hướng đi (bytes)
8	<code>Fwd Packet Length Min</code>	Kích thước gói tin nhỏ nhất theo hướng đi (bytes)
9	<code>Fwd Packet Length Mean</code>	Kích thước gói tin trung bình theo hướng đi (bytes)
10	<code>Fwd Packet Length Std</code>	Độ lệch chuẩn kích thước gói tin theo hướng đi
11	<code>Bwd Packet Length Max</code>	Kích thước gói tin lớn nhất theo hướng về (bytes)
12	<code>Bwd Packet Length Min</code>	Kích thước gói tin nhỏ nhất theo hướng về (bytes)
13	<code>Bwd Packet Length Mean</code>	Kích thước gói tin trung bình theo hướng về (bytes)
14	<code>Bwd Packet Length Std</code>	Độ lệch chuẩn kích thước gói tin theo hướng về
15	<code>Flow Bytes/s</code>	Tốc độ truyền tải byte trên một đơn vị giây

STT	Tên đặc trưng trong dữ liệu	Ý nghĩa giao thức và thống kê luồng mạng
16	Flow Packets/s	Tốc độ truyền tải gói tin trên một đơn vị giây
17	Flow IAT Mean	Thời gian trễ trung bình giữa các gói tin liên tiếp của flow
18	Flow IAT Std	Độ lệch chuẩn thời gian trễ giữa các gói tin của flow
19	Flow IAT Max	Thời gian trễ lớn nhất giữa hai gói tin liên tiếp của flow
20	Flow IAT Min	Thời gian trễ nhỏ nhất giữa hai gói tin liên tiếp của flow
21	Fwd IAT Total	Tổng thời gian trễ giữa các gói tin theo hướng đi
22	Fwd IAT Mean	Thời gian trễ trung bình giữa các gói tin theo hướng đi
23	Fwd IAT Std	Độ lệch chuẩn thời gian trễ giữa các gói tin theo hướng đi
24	Fwd IAT Max	Thời gian trễ lớn nhất giữa các gói tin theo hướng đi
25	Fwd IAT Min	Thời gian trễ nhỏ nhất giữa các gói tin theo hướng đi
26	Bwd IAT Total	Tổng thời gian trễ giữa các gói tin theo hướng về
27	Bwd IAT Mean	Thời gian trễ trung bình giữa các gói tin theo hướng về
28	Bwd IAT Std	Độ lệch chuẩn thời gian trễ giữa các gói tin theo hướng về
29	Bwd IAT Max	Thời gian trễ lớn nhất giữa các gói tin theo hướng về
30	Bwd IAT Min	Thời gian trễ nhỏ nhất giữa các gói tin theo hướng về
31	Fwd PSH Flags	Số lần cờ PSH được bật theo hướng đi
32	Fwd Header Length	Tổng độ dài phần header của các gói tin hướng đi (bytes)
33	Bwd Header Length	Tổng độ dài phần header của các gói tin hướng về (bytes)
34	Fwd Packets/s	Tốc độ truyền gói tin hướng đi trên mỗi giây
35	Bwd Packets/s	Tốc độ truyền gói tin hướng về trên mỗi giây
36	Min Packet Length	Kích thước gói tin nhỏ nhất quan sát được trong toàn bộ flow
37	Max Packet Length	Kích thước gói tin lớn nhất quan sát được trong toàn bộ flow
38	Packet Length Mean	Độ dài trung bình của tất cả các gói tin trong flow

STT	Tên đặc trưng trong dữ liệu	Ý nghĩa giao thức và thống kê luồng mạng
39	Packet Length Std	Độ lệch chuẩn kích thước toàn bộ các gói tin trong flow
40	Packet Length Variance	Phương sai phân bố kích thước các gói tin trong flow
41	FIN Flag Count	Số lượng cờ FIN xuất hiện trong flow (báo hiệu đóng kết nối)
42	SYN Flag Count	Số lượng cờ SYN xuất hiện trong flow (báo hiệu bắt tay TCP)
43	RST Flag Count	Số lượng cờ RST xuất hiện trong flow (thiết lập lại kết nối)
44	PSH Flag Count	Số lượng cờ PSH xuất hiện trong flow (đẩy dữ liệu lên tầng app)
45	ACK Flag Count	Số lượng cờ ACK xuất hiện trong flow (xác nhận gói tin)
46	URG Flag Count	Số lượng cờ URG xuất hiện trong flow (dữ liệu khẩn cấp)
47	ECE Flag Count	Số lượng cờ ECE xuất hiện trong flow (thông báo nghẽn mạng)
48	Down/Up Ratio	Tỷ lệ lưu lượng tải xuống so với tải lên
49	Average Packet Size	Kích thước gói tin trung bình tổng thể của flow
50	Avg Fwd Segment Size	Kích thước phân đoạn TCP trung bình theo hướng đi
51	Avg Bwd Segment Size	Kích thước phân đoạn TCP trung bình theo hướng về
52	Subflow Fwd Packets	Số lượng gói tin trung bình trong một subflow hướng đi
53	Subflow Fwd Bytes	Số lượng byte trung bình trong một subflow hướng đi
54	Subflow Bwd Packets	Số lượng gói tin trung bình trong một subflow hướng về
55	Subflow Bwd Bytes	Số lượng byte trung bình trong một subflow hướng về
56	Init_Win_bytes_forward	Kích thước cửa sổ TCP nhận ban đầu phía client (bytes)
57	Init_Win_bytes_backward	Kích thước cửa sổ TCP nhận ban đầu phía server (bytes)
58	act_data_pkt_fwd	Số lượng gói tin hướng đi thực sự có mang payload dữ liệu
59	min_seg_size_forward	Kích thước phân đoạn tối thiểu được quan sát phía forward
60	Active Mean	Thời gian hoạt động trung bình trước khi chuyển sang trạng thái chờ
61	Active Std	Độ lệch chuẩn thời gian hoạt động của flow
62	Active Max	Thời gian hoạt động tối đa của flow

STT	Tên đặc trưng trong dữ liệu	Ý nghĩa giao thức và thống kê luồng mạng
63	Active Min	Thời gian hoạt động tối thiểu của flow
64	Idle Mean	Thời gian chờ (không có traffic) trung bình của flow
65	Idle Std	Độ lệch chuẩn thời gian chờ của flow
66	Idle Max	Thời gian chờ tối đa của flow
67	Idle Min	Thời gian chờ tối thiểu của flow

Phụ lục D

QUY TẮC RỄ NHÁNH CỦA CÂY QUYẾT ĐỊNH

Dưới đây là cấu trúc logic và các quy tắc phân loại văn bản được trích xuất trực tiếp từ mô hình Decision Tree huấn luyện trên tập Train theo thời gian trong kịch bản Bóc tách Cổng mạng (`without_port`). Khi không có đặc trưng Destination Port, mô hình buộc phải rẽ nhánh dựa trên hình thái thống kê gói tin với nút gốc là Average Packet Size ≤ 14.12 (đối chiếu với cấu trúc cây kịch bản Có cổng tại Mục 6.6.2):

Listing D.1: Toàn văn quy tắc rẽ nhánh cấp cao của Cây Quyết Định (`max_depth=4`)

```
# Top Decision Rules for DecisionTree (max_depth=4 preview)
# Total Depth: 9, Leaves: 17

|--- Average Packet Size <= 14.12
|   |--- Average Packet Size <= 9.29
|       |--- SYN Flag Count <= 0.50
|           |--- Total Length of Bwd Packets <= 57.00
|               |--- Flow IAT Min <= 36509.50
|                   |--- class: 0 (Normal)
|                       |--- Flow IAT Min > 36509.50
|                           |--- truncated branch of depth 5
|                               |--- Total Length of Bwd Packets > 57.00
|                                   |--- class: 1 (Attack)
|                                       |--- SYN Flag Count > 0.50
|                                           |--- Fwd Packet Length Mean <= 5.25
|                                               |--- class: 0 (Normal)
|                                                   |--- Fwd Packet Length Mean > 5.25
|                                                       |--- class: 1 (Attack)
|   |--- Average Packet Size > 9.29
|       |--- Packet Length Std <= 14.32
|           |--- Fwd Packet Length Std <= 14.42
|               |--- Flow IAT Max <= 1.50
|                   |--- truncated branch of depth 4
|                       |--- Flow IAT Max > 1.50
|                           |--- class: 1 (Attack)
|                               |--- Fwd Packet Length Std > 14.42
```

```
| | | | |--- class: 0 (Normal)
| | |--- Packet Length Std > 14.32
| | | |--- class: 0 (Normal)
|--- Average Packet Size > 14.12
| |--- class: 0 (Normal)
```